

Document number: P0958R3
Date: 2021-02-06
Project: Programming Language C++
Audience: SG4 - Networking
Reply-to: Christopher Kohlhoff <chris@kohlhoff.com>

Networking TS changes to support proposed Executors TS

Introduction

The purpose of this paper is to illustrate the changes to the Networking TS to conform to the proposed Executors TS in P0443R13.

All changes are relative to N4711.

Summary of changes

This paper proposes the following changes to the Networking TS:

- [Add a reference to the executors proposal.](#)
- [Remove `executor\_work\_guard` and make `work\_guard`, as these have been superseded by the executors proposal's `execution::outstanding\_work\_property`.](#)
- [Remove the `Executor` type requirements and the `is\_executor` type trait.](#)
- [Specify the "Requirements on asynchronous operations" in terms of the new executors model.](#)
- [Update `system\_executor` and `system\_context` to conform to the new executors model.](#)
- [Remove the polymorphic wrapper `executor` and uses of it as a default executor.](#) As a consequence of this change:
 - The `basic_` prefixes have been removed from the timer, socket, and resolver class templates, as users are now likely to utilise these directly.
 - Type aliases that relied on a default executor (such as `X::socket`) have been replaced with template type aliases taking an executor (such as `X::socket_for`).
- [Specify the free functions `dispatch`, `post`, and `defer` in terms of the new executors model.](#)
- [Update the `strand` adapter to conform to the new executors model.](#)
- [Update the `use\_future\_completion` token to conform to the new executors model.](#)
- [Update `io\_context` to conform to the new executors model.](#)

Deployment experience

An implementation of the changes below, including a complete implementation of the executors-related specifications P0443R13, P1348R0, and P1393R0, can be found in the Asio library at <https://github.com/chriskohlhoff/asio> and in Boost 1.74.

This implementation has been used to recompile libraries that depend on Asio, and has been tested in a number of applications. Some library modifications were required, but the majority of the applications needed minor or no modifications.

Proposed wording

Add a reference to the executors proposal

Add a reference to the executors proposal in -5- Namespaces and headers [namespaces]:

-2- Unless otherwise specified, references to other entities described in this Technical Specification are assumed to be qualified with `std::experimental::net::v1::`, references to entities described in the C++ standard are assumed to be qualified with `std::`, **and** references to entities described in C++ Extensions for Library Fundamentals are assumed to be qualified with `std::experimental::fundamentals_v2::`, **and references to entities described in P1393R0 A General Property Customization Mechanism and in P0443R13 A Unified Executors Proposal for C++ are assumed to be qualified with `std::`.**

Remove superseded `executor_work_guard` and make `work_guard`

Remove `executor_work_guard` from -12.1- Header <experimental/netfwd> synopsis [fwd.decl.synop]:

```
template<class Executor>  
class executor_work_guard;
```

Remove `executor_work_guard` and `make_work_guard` from -13.1- Header `<experimental/executor>` synopsis [async.synop]:

```
template<class Executor>  
class executor_work_guard;  
  
// 13.17, make_work_guard:  
  
template<class Executor>  
executor_work_guard<Executor>  
make_work_guard(const Executor& ex);  
template<class ExecutionContext>  
executor_work_guard<typename ExecutionContext::executor_type>  
make_work_guard(ExecutionContext& ctx);  
template<class T>  
executor_work_guard<associated_executor_t<T>>  
make_work_guard(const T& t);  
template<class T, class U>  
auto make_work_guard(const T& t, U& u)  
> decltype(make_work_guard(get_associated_executor(t, forward<U>(u))));
```

Remove sections -13.16- Class `template executor_work_guard` [async.exec.work.guard] and -13.17- Function `make_work_guard` [async.make.work.guard] in their entirety.

Remove Executor requirements and `is_executor` type trait

Modify Table 3 - Template parameters and type requirements [summary] as follows:

Executor	executor (13.2.2) <u>execution::executor concept (P0443R13).</u>
----------	--

Remove `is_executor` and `is_executor_v` from -13.1- Header `<experimental/executor>` synopsis [async.synop]:

```
template<class T> struct is_executor;  
  
template<class T>  
constexpr bool is_executor_v = is_executor<T>::value;
```

Remove section -13.2.2- Executor requirements [async.reqmts.executor] in its entirety.

Modify Table 5 - ExecutionContext requirements [async.reqmts.executioncontext] as follows:

expression	return type	assertion/note pre/post-condition
<code>X::executor_type</code>	type meeting Executor (13.2.2) requirements <u>A type that models execution::executor (P0443R13).</u>	

Modify section -13.2.7.8- I/O executor [async.reqmts.async.io.exec] as follows.

-1- An asynchronous operation has an associated executor ~~satisfying the Executor (13.2.2) requirements~~whose type models execution::executor (P0443R13). If not otherwise specified by the asynchronous operation, this associated executor is an object of type `system_executor`.

[...]

-3- Let `Executor1` be the type of the associated executor. Let `ex1` be a value of type `Executor1`, representing the associated executor object obtained as described above. can_query v<Executor1, execution::context t> shall be true, and std::query(ex1, execution::context t) shall yield a value of type execution_context& or of type E&, where E satisfies the ExecutionContext (13.2.3) requirements.

Modify section -13.2.7.9- Completion handler executor [async.reqmts.async.handler.exec] as follows.

-1- A completion handler object of type CompletionHandler has an associated executor ~~satisfying the Executor requirements (13.2.2)~~~~whose type models execution::executor (P0443R13).~~ The type of this associated executor is associated_executor_t<CompletionHandler, Executor1>. Let Executor2 be the type associated_executor_t<CompletionHandler, Executor1>. Let ex2 be a value of type Executor2 obtained by performing get_associated_executor(completion_handler, ex1). ~~can_query_v<Executor2, execution::context_t> shall be true, and std::query(ex2, execution::context_t) shall yield a value of type execution_context_t or of type E, where E satisfies the ExecutionContext (13.2.3) requirements.~~

Modify section -13.2.7.14- *Composed asynchronous operations [async.reqmts.async.composed]* as follows.

An intermediate operation's completion handler shall have an associated executor that is either:

- the type Executor2 and object ex2 obtained from the completion handler type CompletionHandler and object completion_handler; or
- an object of an unspecified type ~~satisfying the Executor requirements (13.2.2)~~~~that models execution::executor (P0443R13),~~ that delegates executor operations to the type Executor2 and object ex2.

Remove section -13.9- *Class template is_executor [async.is.exec]* in its entirety.

Modify section -13.10- *Executor argument tag [async.executor.arg]* as follows.

The executor_arg_t struct is an empty structure type used as a unique type to disambiguate constructor and function overloading. Specifically, types may have constructors with executor_arg_t as the first argument, immediately followed by an argument of a type that ~~satisfies the Executor requirements (13.2.2)~~models ~~execution::executor (P0443R13).~~

Modify section -13.12- *Class template associated_executor [async.assoc.exec]* as follows.

-1- Class template associated_executor is an associator (13.2.6) for ~~the Executor (13.2.2) type requirements~~~~executors,~~ with default candidate type system_executor and default candidate object system_executor().

Modify *Table 9 - associated_executor specialization requirements* as follows:

Expression	Return type	Note
typename X::type	A type meeting Executor requirements (13.2.2) A type that models execution::executor (P0443R13).	

Modify section -13.13- *Function get_associated_executor [async.assoc.exec.get]* as follows.

-3- **Remarks:** This function shall not participate in overload resolution unless ~~is_executor_v<Executor> is true~~~~is_convertible<Executor&, execution_context_t>::value is false.~~

Modify section -13.14- *Class template executor_binder [async.exec.binder]* as follows.

-1- The class template executor_binder binds executors to objects. A specialization executor_binder<T, Executor> binds an executor of type Executor ~~satisfying the Executor requirements (13.2.2)~~~~that models execution::executor (P0443R13)~~ to an object or function of type T.

Modify section -13.15- *Function bind_executor [async.bind.executor]* as follows.

-2- **Remarks:** This function shall not participate in overload resolution unless ~~is_executor_v<Executor> is true~~~~is_convertible<Executor&, execution_context_t>::value is false.~~

Modify section -13.22- *Function dispatch [async.dispatch]* as follows.

-8- **Remarks:** This function shall not participate in overload resolution unless ~~is_executor_v<Executor> is true~~~~is_convertible<Executor&, execution_context_t>::value is false.~~

Modify section -13.23- *Function post [async.post]* as follows.

-8- **Remarks:** This function shall not participate in overload resolution unless ~~is_executor_v<Executor> is true~~~~is_convertible<Executor&, execution_context_t>::value is false.~~

Modify section -13.24- *Function defer [async.defer]* as follows.

-8- **Remarks:** This function shall not participate in overload resolution unless ~~is_executor_v<Executor> is true~~is_convertible<Executor&, execution_context&>::value is false.

Modify section -13.25- *Class template strand [async.strand]* as follows.

-1- The class template strand is a wrapper around an object of type Executor ~~satisfying the Executor requirements (13.2.2)~~that models execution::executor (P0443R13).

[...]

-2- strand<Executor> ~~satisfies the Executor (13.2.2) requirements~~models execution::executor (P0443R13).

Modify Table 17 - AsyncReadStream requirements [buffer.stream.reqmts.asyncreadstream] as follows:

operation	type	semantics, pre/post-conditions
a.get_executor()	A type satisfying the Executor requirements (13.2.2) <u>that models execution::executor (P0443R13)</u> .	Returns the associated I/O executor.

Modify Table 19 - AsyncWriteStream requirements [buffer.stream.reqmts.asyncwritestream] as follows:

operation	type	semantics, pre/post-conditions
a.get_executor()	A type satisfying the Executor requirements (13.2.2) <u>that models execution::executor (P0443R13)</u> .	Returns the associated I/O executor.

Specify the "Requirements on asynchronous operations" in terms of the new executors model

Modify section -13.2.7.10- *Outstanding work [async.reqmts.async.work]* as follows.

- 1- Until the asynchronous operation has completed, the asynchronous operation shall maintain:
- ~~an object work1 of type executor_work_guard<Executor1>, initialized as work1(ex1), and where work1.owns_work() == true; and~~
 - ~~an object work2 of type executor_work_guard<Executor2>, initialized as work2(ex2), and where work2.owns_work() == true;~~
 - an executor object work1, initialized as std::prefer(ex1, execution::outstanding_work.tracked); and
 - an executor object work2, initialized as std::prefer(ex2, execution::outstanding_work.tracked).

Modify section -13.2.7.12- *Execution of completion handler on completion of asynchronous operation [async.reqmts.async.completion]* as follows:

-3- If an asynchronous operation completes immediately (that is, within the thread of execution calling the initiating function, and before the initiating function returns), the completion handler shall be submitted for execution as if by performing ~~ex2_post(std::move(f), alloc2)~~. ~~Otherwise, the completion handler shall be submitted for execution as if by performing ex2_dispatch(std::move(f), alloc2);~~

```
execution::execute(
    std::prefer(
        std::require(ex2, execution::blocking.never),
        execution::allocator(alloc2)),
    std::move(f));
```

Otherwise, the completion handler shall be submitted for execution as if by performing:

```
execution::execute(
    std::prefer(work2, execution::blocking.possibly),
    execution::allocator(alloc2)),
    std::move(f));
```

Update system_executor and system_context to conform to the new executors model

Remove `system_executor` from, and add `system_context` to, -12.1- Header <experimental/netfwd> synopsis [fwd.decl.synop]:

```
class system_executor;  
class system_context;
```

Update `system_executor` to be a type alias in -13.1- Header <experimental/executor> synopsis [async.synop]:

```
class system_executor;  
class system_context;  
using system_executor = system_context::executor_type;  
  
bool operator==(const system_executor&, const system_executor&);  
bool operator!=(const system_executor&, const system_executor&);
```

Remove section -13.18- Class `system_executor` [async.system.exec] in its entirety.

Modify section -13.19- Class `system_context` [async.system.context] as follows:

-1- Class `system_context` implements ~~the execution context associated with `system_executor` objects~~ an execution context that represents the ability to run a submitted function object on any thread.

```
namespace std {  
namespace experimental {  
namespace net {  
inline namespace v1 {  
  
    class system_context : public execution_context  
    {  
    public:  
        // types:  
  
        using executor_type = system_executor see below;
```

[...]

-2- The class `system_context` satisfies the `ExecutionContext` (13.2.3) type requirements.

-?- `executor_type` is an executor type conforming to the specification for `system_context` executor types described below. Executor objects of type `executor_type` have the following properties established:

- `execution::blocking.possibly`
- `execution::relationship.fork`
- `execution::mapping.thread`
- `execution::allocator(std::allocator<void>())`

-?- `system_context` executors having a different set of established properties may be represented by distinct, unspecified types. Function objects submitted via a `system_context` executor object are permitted to execute on any thread. To satisfy the requirements for the `execution::blocking.never` property, a `system_context` executor may create thread objects to run the submitted function objects. These thread objects are collectively referred to as system threads.

[...]

```
executor_type get_executor() noexcept;
```

-5- **Returns:** ~~`system_executor()`~~ `executor_type()`.

After section -13.19- Class `system_context` [async.system.context] insert a new section as follows:

-13.-? system context executor types

-1- Class `system-context-executor` is for exposition only. It is used to specify a bounded set of types that model `execution::executor` (P0443R13). All executor types accessible through `system_context::executor_type`, `system_context::get_executor()`, and subsequent calls to the member function require, are members of this set and conform to the specification of `system-context-executor`. [Note: An implementation may provide distinct types for executors that have different properties established. --end note]

```
namespace std {  
namespace experimental {
```

```

namespace net {
inline namespace v1 {

    class system-context-executor
    {
    public:
        // construct / copy / destroy:

        system-context-executor() {}.

        // executor operations:

        see below require(execution::blocking t::possibly t) const;
        see below require(execution::blocking t::never t) const;
        see below require(execution::blocking t::always t) const;
        see below require(execution::relationship t::fork t) const;
        see below require(execution::relationship t::continuation t) const;
        see below require(execution::allocator t<void>) const;
        template<class ProtoAllocator>
        see below require(const execution::allocator t<ProtoAllocator>& a) const;

        static constexpr execution::mapping_t query(execution::mapping_t) noexcept;
        system_context& query(execution::context_t) const noexcept;
        execution::blocking_t query(execution::blocking_t) const noexcept;
        execution::relationship_t query(execution::relationship_t) const noexcept;
        see below query(execution::allocator t<void>) const noexcept;
        template<class ProtoAllocator>
        see below query(const execution::allocator t<ProtoAllocator>&) const noexcept;

        template<class Function>
        void execute(Function&& f) const;
    };

    bool operator==(const system-context-executor& a, const system-context-executor& b) noexcept;
    bool operator!=(const system-context-executor& a, const system-context-executor& b) noexcept;

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

-13.?.1- system context executor operations

see below require(execution::blocking t::possibly t) const;
see below require(execution::blocking t::never t) const;
see below require(execution::blocking t::always t) const;
see below require(execution::relationship t::fork t) const;
see below require(execution::relationship t::continuation t) const;

-1- Returns: An object of a *system-context-executor* class, with the requested property established. When the requested property is part of a group that is defined as a mutually exclusive set, any other properties in the group are removed from the returned executor object. All other properties of the returned executor object are identical to those of **this*.

see below require(execution::allocator t<void>) const;

-2- Returns: require(execution::allocator(std::allocator<void>()).).

template<class ProtoAllocator>
see below require(const execution::allocator t<ProtoAllocator>& a) const;

-3- Returns: An object of a *system-context-executor* class, with the execution::allocator t<ProtoAllocator> property established such that allocation and deallocation associated with function submission will be performed using a copy of a.value(). All other properties of the returned executor object are identical to those of **this*.

static constexpr execution::mapping_t query(execution::mapping_t) noexcept;

-4- Returns: true.

system_context& query(execution::context_t) const;

-5- Returns: A reference to the system_context object.

execution::blocking_t query(execution::blocking_t) const noexcept;
execution::relationship_t query(execution::relationship_t) const noexcept;

-6- Returns: The established value of the property for the executor object *this.

see below query(execution::allocator t<void>) const noexcept;
template<class ProtoAllocator>
see below query(const execution::allocator t<ProtoAllocator>&) const noexcept;

-7- Returns: The allocator object associated with the executor, with type and value as previously established by the execution::allocator t<ProtoAllocator> property.

template<class Function>
void execute(Function&& f) const

-8- Effects: Submits the function f for execution according to the execution::executor concept and the properties established for *this. If f exits via an exception, calls std::terminate().

-13.2- system context executor comparisons

bool operator==(const system-context-executor& a, const system-context-executor& b) noexcept;

-1- Returns: true if a and b have identical properties, otherwise false.

bool operator!=(const system-context-executor& a, const system-context-executor& b) noexcept;

-2- Returns: !(a == b).

Remove the polymorphic wrapper executor and uses of it as a default executor

Remove executor from -12.1- Header <experimental/netfwd> synopsis [fwd.decl.synop]:

~~class executor;~~

Assuming the changes in P1322 have been applied,

- remove executor as the default for Executor template parameters,
- use io_context::executor_type as the default for the iostream class templates' Executor template parameters,
- reorder template parameters so that the remaining defaulted ones come last,
- remove the basic_ prefixes, and
- remove the system_timer, steady_timer, and high_resolution_timer type aliases

in -12.1- Header <experimental/netfwd> synopsis [fwd.decl.synop]:

```
template<class Clock> struct wait_traits;
template<class Clock, class Executor, class WaitTraits = wait_traits<Clock>, class Executor = executor>
    class basic_waitable_timer;
using system_timer = basic_waitable_timer<chrono::system_clock>;
using steady_timer = basic_waitable_timer<chrono::steady_clock>;
using high_resolution_timer = basic_waitable_timer<chrono::high_resolution_clock>;

template<class Protocol, class Executor = executor>
    class basic_socket;
template<class Protocol, class Executor = executor>
    class basic_datagram_socket;
template<class Protocol, class Executor = executor>
    class basic_stream_socket;
template<class Protocol, class Executor = executor>
    class basic_socket_acceptor;
template<class Protocol, class Clock = chrono::steady_clock,
    class WaitTraits = wait_traits<Clock>, class Executor = executorio_context::executor_type>
    class basic_socket_streambuf;
template<class Protocol, class Clock = chrono::steady_clock,
    class WaitTraits = wait_traits<Clock>, class Executor = executorio_context::executor_type>
    class basic_socket_iostream;

namespace ip {
```


[...]

```
template<class InternetProtocol, class Executor = executor>
    class basic_resolver;
```

Remove classes `bad_executor` and `executor` from -13.1- Header `<experimental/executor>` synopsis [async.synop]:

```
class bad_executor;

class executor;

bool operator==(const executor& a, const executor& b) noexcept;
bool operator==(const executor& e, nullptr_t) noexcept;
bool operator==(nullptr_t, const executor& e) noexcept;
bool operator!=(const executor& a, const executor& b) noexcept;
bool operator!=(const executor& e, nullptr_t) noexcept;
bool operator!=(nullptr_t, const executor& e) noexcept;
```

[...]

```
} // inline namespace v1
} // namespace net
} // namespace experimental

template<class Allocator>
struct uses_allocator<experimental::net::v1::executor, Allocator>
: true_type {};

} // namespace std
```

Remove sections -13.20- Class `bad_executor` [async.bad.exec] and -13.21- Class `executor` [async.executor] in their entirety.

Use the `waitable_timer` class template directly, instead of the `steady_timer` alias, in -15- Timers [timer]:

-1- This clause defines components for performing timer operations.

-2- [Example: Performing a synchronous wait operation on a timer:

```
io_context c;
steady_timerwaitable_timer<chrono::steady_clock> t(c);
t.expires_after(seconds(5));
t.wait();
```

-- end example]

-3- [Example: Performing an asynchronous wait operation on a timer:

```
void handler(error_code ec) { ... }
...
io_context c;
steady_timerwaitable_timer<chrono::steady_clock> t(c);
t.expires_after(seconds(5));
t.async_wait(handler);
c.run();
```

-- end example]

Assuming the changes in P1322 have been applied,

- remove `executor` as the default for the `Executor` template parameter,
- reorder the template parameters so that the remaining defaulted ones come last, and
- remove the `basic_` prefix, and
- remove the `system_timer`, `steady_timer`, and `high_resolution_timer` type aliases

from -15.1- Header `<experimental/timer>` synopsis [timer.synop]

```
#include <chrono>

namespace std {
```



```

namespace experimental {
namespace net {
inline namespace v1 {

    template<class Clock> struct wait_traits;

    template<class Clock, class Executor, class WaitTraits = wait_traits<Clock>, class Executor = executor>
        class basic_waitable_timer;

    using system_timer = basic_waitable_timer<chrono::system_clock>;
    using steady_timer = basic_waitable_timer<chrono::steady_clock>;
    using high_resolution_timer = basic_waitable_timer<chrono::high_resolution_clock>;

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

Remove the `basic_` prefix from `basic_waitable_timer` in -15.2.1- Wait traits requirements [timer.reqmts.waittraits] as follows:

-1- The `basic_waitable_timer` template uses wait traits to allow programs to customize wait and `async_wait` behavior. [Note: Possible uses of wait traits include:

Assuming the changes in P1322 have been applied,

- remove `executor` as the default for the `Executor` template parameter,
- reorder the template parameters so that the remaining defaulted ones come last, and
- remove the `basic_` prefix

from -15.4- Class template `basic_waitable_timer` [timer.waitable]

15.4 Class template `basic_waitable_timer` [timer.waitable]

```

namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

    template<class Clock, class Executor, class WaitTraits = wait_traits<Clock>, class Executor = executor>
        class basic_waitable_timer
        {
        public:
            // types:

            using executor_type = Executor;
            using clock_type = Clock;
            using duration = typename clock_type::duration;
            using time_point = typename clock_type::time_point;
            using traits_type = WaitTraits;

            template<class OtherExecutor>
            using rebind_executor =
                basic_waitable_timer<Clock, WaitTraits, OtherExecutor>;

            // [timer.waitable.cons], construct / copy / destroy:

            explicit basic_waitable_timer();
            template<class ExecutionContext>
                explicit basic_waitable_timer(ExecutionContext& ctx);
            basic_waitable_timer(const executor_type& ex, const time_point& t);
            template<class ExecutionContext>
                basic_waitable_timer(ExecutionContext& ctx, const time_point& t);
            basic_waitable_timer(const executor_type& ex, const duration& d);
            template<class ExecutionContext>
                basic_waitable_timer(ExecutionContext& ctx, const duration& d);
            basic_waitable_timer(const basic_waitable_timer&) = delete;
            basic_waitable_timer(basic_waitable_timer&& rhs);

            ~basic_waitable_timer();

            basic_waitable_timer& operator=(const basic_waitable_timer&) = delete;

```

```

basic_waitable_timer& operator=(basic_waitable_timer&& rhs);

// [timer.waitable.ops], basic_waitable_timer operations:

executor_type get_executor() noexcept;

size_t cancel();
size_t cancel_one();

time_point expiry() const;
size_t expires_at(const time_point& t);
size_t expires_after(const duration& d);

void wait();
void wait(error_code& ec);

template<class CompletionToken>
    DEDUCED async_wait(CompletionToken&& token);
};

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

-1- Instances of class template ~~basic~~_waitable_timer meet the requirements of Destructible (C++2014[destructible]), MoveConstructible (C++2014[moveconstructible]), and MoveAssignable (C++2014[moveassignable]).

15.4.1 ~~basic~~_waitable_timer constructors [timer.waitable.cons]

```
explicit basic_waitable_timer(const executor_type& ex);
```

-1- *Effects*: Equivalent to ~~basic~~_waitable_timer(ex, time_point()).

```
template<class ExecutionContext>
    explicit basic_waitable_timer(ExecutionContext& ctx);
```

Effects: Equivalent to ~~basic~~_waitable_timer(ctx.get_executor()).

Remarks: This function shall not participate in overload resolution unless is_convertible<ExecutionContext&, execution_context&>::value is true and is_constructible<executor_type, typename ExecutionContext::executor_type>::value is true.

```
basic_waitable_timer(const executor_type& ex, const time_point& t);
```

-2- *Postconditions*:

- (2.1) get_executor() == ex.
- (2.2) expiry() == t.

```
template<class ExecutionContext>
    basic_waitable_timer(ExecutionContext& ctx, const time_point& t);
```

Effects: Equivalent to ~~basic~~_waitable_timer(ctx.get_executor(), t).

Remarks: This function shall not participate in overload resolution unless is_convertible<ExecutionContext&, execution_context&>::value is true and is_constructible<executor_type, typename ExecutionContext::executor_type>::value is true.

```
basic_waitable_timer(const executor_type& ex, const duration& d);
```

-3- *Effects*: Sets the expiry time as if by calling expires_after(d).

-4- *Postconditions*: get_executor() == ex.

```
template<class ExecutionContext>
    basic_waitable_timer(ExecutionContext& ctx, const duration& d);
```

Effects: Equivalent to ~~basic~~_waitable_timer(ctx.get_executor(), d).

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context&>::value` is true and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

```
basic_waitable_timer(basic_waitable_timer&& rhs);
```

-5- *Effects:* Move constructs an object of class ~~`basic_waitable_timer`~~<Clock, waitTraits> that refers to the state originally represented by rhs.

-6- *Postconditions:*

- (6.1) `get_executor()` == `rhs.get_executor()`.
- (6.2) `expiry()` returns the same value as `rhs.expiry()` prior to the constructor invocation.
- (6.3) `rhs.expiry()` == `time_point()`.

15.4.2 ~~`basic_waitable_timer`~~ destructor [`timer.waitable.dtor`]

```
~basic_waitable_timer();
```

-1- *Effects:* Destroys the timer, canceling any asynchronous wait operations associated with the timer as if by calling `cancel()`.

15.4.3 ~~`basic_waitable_timer`~~ assignment [`timer.waitable.assign`]

```
basic_waitable_timer& operator=(basic_waitable_timer&& rhs);
```

-1- *Effects:* Cancels any outstanding asynchronous operations associated with `*this` as if by calling `cancel()`, then moves into `*this` the state originally represented by rhs.

-2- *Postconditions:*

- (2.1) `get_executor()` == `rhs.get_executor()`.
- (2.2) `expiry()` returns the same value as `rhs.expiry()` prior to the assignment.
- (2.3) `rhs.expiry()` == `time_point()`.

-3- *Returns:* `*this`.

15.4.4 ~~`basic_waitable_timer`~~ operations [`timer.waitable.ops`]

[...]

```
template<class CompletionToken>
    DEDUCED async_wait(CompletionToken&& token);
```

-15- *Completion signature:* `void(error_code ec)`.

-16- *Effects:* Initiates an asynchronous wait operation to repeatedly wait for the relative time produced by `waitTraits::to_wait_duration(e)`, where `e` is a value of type `time_point` such that `e <= expiry()`. The completion handler is submitted for execution only when the condition `ec || expiry() <= clock_type::now()` yields true.

-17- [*Note:* To implement `async_wait`, an `io_context` object `ctx` could maintain a priority queue for each specialization of ~~`basic_waitable_timer`~~<Clock, waitTraits> for which a timer object was initialized with `ctx`. Only the time point `e` of the earliest outstanding expiry need be passed to `waitTraits::to_wait_duration(e)`. -- *end note*]

Assuming the changes in P1322 have been applied,

- remove `executor` as the default for `Executor` template parameters,
- use `io_context::executor_type` as the default for the `iostream` class templates' `Executor` template parameters,
- remove the `basic_` prefixes

in -18.1- Header <experimental/socket> synopsis [`socket.synop`]:

```
namespace std {
namespace experimental {
namespace net {
```

```

inline namespace v1 {

enum class socket_errc {
    already_open = an implementation-defined non-zero value,
    not_found = an implementation-defined non-zero value
};

const error_category& socket_category() noexcept;

error_code make_error_code(socket_errc e) noexcept;
error_condition make_error_condition(socket_errc e) noexcept;

// Sockets:

class socket_base;

template<class Protocol, class Executor == executor>
    class basic_socket;

template<class Protocol, class Executor == executor>
    class basic_datagram_socket;

template<class Protocol, class Executor == executor>
    class basic_stream_socket;

template<class Protocol, class Executor == executor>
    class basic_socket_acceptor;

// [socket.iostreams], Socket streams:

template<class Protocol, class Clock = chrono::steady_clock,
    class WaitTraits = wait_traits<Clock>, class Executor = executorio_context::executor_type>
    class basic_socket_streambuf;

template<class Protocol, class Clock = chrono::steady_clock,
    class WaitTraits = wait_traits<Clock>, class Executor = executorio_context::executor_type>
    class basic_socket_istream;

// [socket.algo.connect], synchronous connect operations:

template<class Protocol, class Executor, class EndpointSequence>
    typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,
        const EndpointSequence& endpoints);
template<class Protocol, class Executor, class EndpointSequence>
    typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,
        const EndpointSequence& endpoints,
        error_code& ec);
template<class Protocol, class Executor, class EndpointSequence, class ConnectCondition>
    typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,
        const EndpointSequence& endpoints,
        ConnectCondition c);
template<class Protocol, class Executor, class EndpointSequence, class ConnectCondition>
    typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,
        const EndpointSequence& endpoints,
        ConnectCondition c,
        error_code& ec);

template<class Protocol, class Executor, class InputIterator>
    InputIterator connect(basic_socket<Protocol, Executor>& s,
        InputIterator first, InputIterator last);
template<class Protocol, class Executor, class InputIterator>
    InputIterator connect(basic_socket<Protocol, Executor>& s,
        InputIterator first, InputIterator last,
        error_code& ec);
template<class Protocol, class Executor, class InputIterator, class ConnectCondition>
    InputIterator connect(basic_socket<Protocol, Executor>& s,
        InputIterator first, InputIterator last,
        ConnectCondition c);
template<class Protocol, class Executor, class InputIterator, class ConnectCondition>
    InputIterator connect(basic_socket<Protocol, Executor>& s,
        InputIterator first, InputIterator last,
        ConnectCondition c,
        error_code& ec);

// [socket.algo.async.connect], asynchronous connect operations:

```

```

template<class Protocol, class Executor, class EndpointSequence, class CompletionToken>
    DEDUCED async_connect(basic_socket<Protocol, Executor>& s,
        const EndpointSequence& endpoints,
        CompletionToken&& token);
template<class Protocol, class Executor, class EndpointSequence, class ConnectCondition,
    class CompletionToken>
    DEDUCED async_connect(basic_socket<Protocol, Executor>& s,
        const EndpointSequence& endpoints,
        ConnectCondition c,
        CompletionToken&& token);

template<class Protocol, class Executor, class InputIterator, class CompletionToken>
    DEDUCED async_connect(basic_socket<Protocol, Executor>& s,
        InputIterator first, InputIterator last,
        CompletionToken&& token);
template<class Protocol, class Executor, class InputIterator, class ConnectCondition,
    class CompletionToken>
    DEDUCED async_connect(basic_socket<Protocol, Executor>& s,
        InputIterator first, InputIterator last,
        ConnectCondition c,
        CompletionToken&& token);

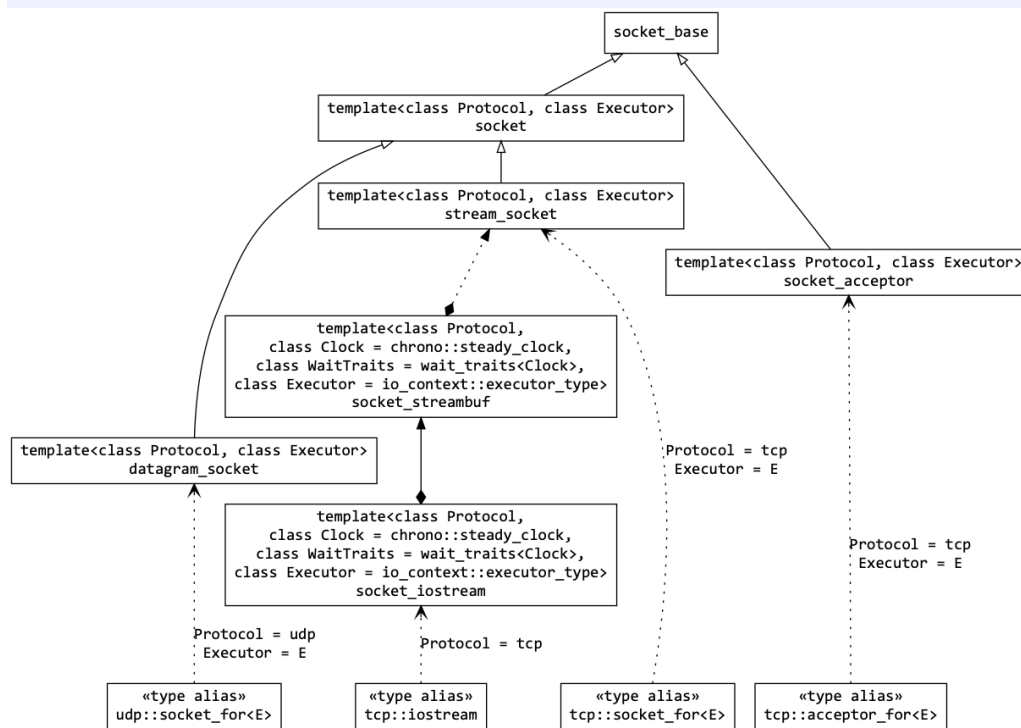
} // inline namespace v1
} // namespace net
} // namespace experimental

template<> struct is_error_code_enum<experimental::net::v1::socket_errc>
    : public true_type {};

} // namespace std

[...]
```

Update Figure 1 "Socket and socket stream types [non-normative]" in -18.1- Header <experimental/socket> synopsis [socket.synop] to the following:



Assuming the changes in P1322 have been applied,

- remove the socket type alias from the AcceptableProtocol requirements,
- refer to the execution::executor concept, and
- remove the basic_ prefixes

from -18.2.7- Acceptable protocol requirements [socket.reqmts.acceptableprotocol] :

-1- A type X meets the AcceptableProtocol requirements if it satisfies the requirements of Protocol ([socket.reqmts.protocol]) as well as the additional requirements listed below.

In the table below, E is a type that satisfies the Executor requirements ([async.reqmts.executor])models execution::executor (P0443R13).

Table 26: AcceptableProtocol requirements

expression	return type	assertion/note pre/post-conditions
X::socket	A type that satisfies the requirements of Destructible (C++2014[destructible]) and MoveConstructible (C++2014[moveconstructible]), and that is publicly and unambiguously derived from basic_socket<X>.	
X::socket_ - for<E>	A type that satisfies the requirements of Destructible (C++2014[destructible]) and MoveConstructible (C++2014[moveconstructible]), and that is publicly and unambiguously derived from basic_ socket<X, E>.	

[...]

Assuming the changes in P1322 have been applied, remove the basic_ prefixes from -18.4- Class socket_base [socket.base]:

[...]

-1- socket_base defines several member types:

- (1.1) socket option classes broadcast, debug, do_not_route, keep_alive, linger, out_of_band_inline, receive_buffer_size, receive_low_watermark, reuse_address, send_buffer_size, and send_low_watermark;
- (1.2) an enumerated type, shutdown_type, for use with the ~~basic_~~socket<Protocol, Executor> class's shutdown member function.
- (1.3) an enumerated type, wait_type, for use with the ~~basic_~~socket<Protocol, Executor> and ~~basic_~~socket_ - acceptor<Protocol, Executor> classes' wait and async_wait member functions,
- (1.4) a bitmask type, message_flags, for use with the ~~basic_~~stream_socket<Protocol, Executor> class's send, async_send, receive, and async_receive member functions, and the ~~basic_~~datagram_socket<Protocol> class's send, async_send, send_to, async_send_to, receive, async_receive, receive_from, and async_receive_from member functions.
- (1.5) a constant, max_listen_connections, for use with the ~~basic_~~socket_acceptor<Protocol, Executor> class's listen member function.

[...]

Assuming the changes in P1322 have been applied, remove the basic_ prefixes from -18.6- Class basic_socket [socket.basic]:

18.6 Class template ~~basic_~~socket [socket.basic]

-1- Class template ~~basic_~~socket<Protocol, Executor> is used as the base class for the ~~basic_~~datagram_socket<Protocol, Executor> and ~~basic_~~stream_socket<Protocol, Executor> class templates. It provides functionality that is common to both types of socket.

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

    template<class Protocol, class Executor>
    class basic_socket : public socket_base
    {
    public:
        // types:

        using executor_type = Executor;
        using native_handle_type = implementation-defined; // see [socket.reqmts.native]
```

```

using protocol_type = Protocol;
using endpoint_type = typename protocol_type::endpoint;

template<class OtherExecutor>
using rebind_executor =
    basic_socket<Protocol, OtherExecutor>;

// [socket.basic.ops], basic_socket operations:

[...]
```

```

protected:
    // [socket.basic.cons], construct / copy / destroy:

    explicit basic_socket(const executor_type& ex);
    template<class ExecutionContext>
        explicit basic_socket(ExecutionContext& ctx);
    basic_socket(const executor_type& ex, const protocol_type& protocol);
    template<class ExecutionContext>
        basic_socket(ExecutionContext& ctx, const protocol_type& protocol);
    basic_socket(const executor_type& ex, const endpoint_type& endpoint);
    template<class ExecutionContext>
        basic_socket(ExecutionContext& ctx, const endpoint_type& endpoint);
    basic_socket(const executor_type& ex, const protocol_type& protocol,
        const native_handle_type& native_socket); // see [socket.reqmts.native]
    template<class ExecutionContext>
        basic_socket(ExecutionContext& ctx, const protocol_type& protocol,
            const native_handle_type& native_socket); // see [socket.reqmts.native]
    basic_socket(const basic_socket&) = delete;
    basic_socket(basic_socket&& rhs);
    template<class OtherProtocol, class OtherExecutor>
        basic_socket(basic_socket<OtherProtocol, OtherExecutor>&& rhs);

    ~basic_socket();

    basic_socket& operator=(const basic_socket&) = delete;
    basic_socket& operator=(basic_socket&& rhs);
    template<class OtherProtocol, class OtherExecutor>
        basic_socket& operator=(basic_socket<OtherProtocol, OtherExecutor>&& rhs);

private:
    protocol_type protocol_; // exposition only
};

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

-2- Instances of class template `basic_socket` meet the requirements of Destructible (C++2014[destructible]), MoveConstructible (C++2014[moveconstructible]), and MoveAssignable (C++2014[moveassignable]).

-3- When an operation has its effects specified as if by passing the result of `native_handle()` to a POSIX function, then the operation fails with error condition `errc::bad_file_descriptor` if `is_open() == false` at the point in the effects when the POSIX function is called.

18.6.1 `basic_socket` constructors [socket.basic.cons]

```
explicit basic_socket(const executor_type& ex);
```

-1- *Postconditions*:

- (1.1) `get_executor() == ex`.
- (1.2) `is_open() == false`.

```
template<class ExecutionContext>
    explicit basic_socket(ExecutionContext& ctx);
```

Effects: Equivalent to `basic_socket(ctx.get_executor())`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>` is true and `is_constructible<executor_type, typename ExecutionContext::executor_type::value>`

is true.

```
basic_socket(const executor_type& ex, const protocol_type& protocol);
```

-2- *Effects*: Opens this socket as if by calling `open(protocol)`.

-3- *Postconditions*:

- (3.1) `get_executor() == ex`.
- (3.2) `is_open() == true`.
- (3.3) `non_blocking() == false`.
- (3.4) `protocol_ == protocol`.

```
template<class ExecutionContext>  
explicit basic_socket(ExecutionContext& ctx, const protocol_type& protocol);
```

Effects: Equivalent to `basic_socket(ctx.get_executor(), protocol)`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>::value` is true and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

```
basic_socket(const executor_type& ex, const endpoint_type& endpoint);
```

-4- *Effects*: Opens and binds this socket as if by calling:

```
open(endpoint.protocol());  
bind(endpoint);
```

-5- *Postconditions*:

- (5.1) `get_executor() == ex`.
- (5.2) `is_open() == true`.
- (5.3) `non_blocking() == false`.
- (5.4) `protocol_ == endpoint.protocol()`.

```
template<class ExecutionContext>  
explicit basic_socket(ExecutionContext& ctx, const endpoint_type& endpoint);
```

Effects: Equivalent to `basic_socket(ctx.get_executor(), endpoint)`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>::value` is true and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

```
basic_socket(const executor_type& ex, const protocol_type& protocol,  
             const native_handle_type& native_socket);
```

-6- *Requires*: `native_socket` is a native handle to an open socket.

-7- *Effects*: Assigns the existing native socket into this socket as if by calling `assign(protocol, native_socket)`.

-8- *Postconditions*:

- (8.1) `get_executor() == ex`.
- (8.2) `is_open() == true`.
- (8.3) `non_blocking() == false`.
- (8.4) `protocol_ == protocol`.

```
template<class ExecutionContext>  
basic_socket(ExecutionContext& ctx, const protocol_type& protocol,  
             const native_handle_type& native_socket);
```

Effects: Equivalent to `basic_socket(ctx.get_executor(), protocol, native_socket)`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>::value` is true and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

```
basic_socket(basic_socket&& rhs);
```

-9- *Effects*: Move constructs an object of class `basic_socket<Protocol, Executor>` that refers to the state originally represented by rhs.

-10- *Postconditions*:

- (10.1) `get_executor()` == `rhs.get_executor()`.
- (10.2) `is_open()` returns the same value as `rhs.is_open()` prior to the constructor invocation.
- (10.3) `non_blocking()` returns the same value as `rhs.non_blocking()` prior to the constructor invocation.
- (10.4) `native_handle()` returns the prior value of `rhs.native_handle()`.
- (10.5) `protocol_` is the prior value of `rhs.protocol_`.
- (10.6) `rhs.is_open()` == `false`.

```
template<class OtherProtocol, class OtherExecutor>  
basic_socket(basic_socket<OtherProtocol, OtherExecutor>&& rhs);
```

-11- *Requires*: `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

-12- *Effects*: Move constructs an object of class `basic_socket<Protocol, Executor>` that refers to the state originally represented by rhs.

-13- *Postconditions*:

- (13.1) `get_executor()` == `rhs.get_executor()`.
- (13.2) `is_open()` returns the same value as `rhs.is_open()` prior to the constructor invocation.
- (13.3) `non_blocking()` returns the same value as `rhs.non_blocking()` prior to the constructor invocation.
- (13.4) `native_handle()` returns the prior value of `rhs.native_handle()`.
- (13.5) `protocol_` is the result of converting the prior value of `rhs.protocol_`.
- (13.6) `rhs.is_open()` == `false`.

-14- *Remarks*: This constructor shall not participate in overload resolution unless `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

18.6.2 `basic_socket` destructor [socket.basic.dtor]

```
~basic_socket();
```

-1- *Effects*: If `is_open()` is true, cancels all outstanding asynchronous operations associated with this socket, disables the linger socket option to prevent the destructor from blocking, and releases socket resources as if by POSIX `close(native_handle())`. Completion handlers for canceled operations are passed an error code `ec` such that `ec == errc::operation_canceled` yields true.

18.6.3 `basic_socket` assignment [socket.basic.assign]

```
basic_socket& operator=(basic_socket&& rhs);
```

-1- *Effects*: If `is_open()` is true, cancels all outstanding asynchronous operations associated with this socket. Completion handlers for canceled operations are passed an error code `ec` such that `ec == errc::operation_canceled` yields true. Disables the linger socket option to prevent the assignment from blocking, and releases socket resources as if by POSIX `close(native_handle())`. Moves into `*this` the state originally represented by rhs.

-2- *Postconditions*:

- (2.1) `get_executor()` == `rhs.get_executor()`.
- (2.2) `is_open()` returns the same value as `rhs.is_open()` prior to the assignment.
- (2.3) `non_blocking()` returns the same value as `rhs.non_blocking()` prior to the assignment.
- (2.4) `protocol_` is the prior value of `rhs.protocol_`.
- (2.5) `rhs.is_open()` == `false`.

-3- *Returns*: `*this`.

```
template<class OtherProtocol, class OtherExecutor>  
basic_socket& operator=(basic_socket<OtherProtocol, OtherExecutor>&& rhs);
```

-4- *Requires:* `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

-5- *Effects:* If `is_open()` is true, cancels all outstanding asynchronous operations associated with this socket. Completion handlers for canceled operations are passed an error code `ec` such that `ec == errc::operation_canceled` yields true. Disables the linger socket option to prevent the assignment from blocking, and releases socket resources as if by POSIX `close(native_handle())`. Moves into `*this` the state originally represented by `rhs`.

-6- *Postconditions:*

- (6.1) `get_executor() == rhs.get_executor()`.
- (6.2) `is_open()` returns the same value as `rhs.is_open()` prior to the assignment.
- (6.3) `non_blocking()` returns the same value as `rhs.non_blocking()` prior to the assignment.
- (6.4) `protocol_` is the result of converting the prior value of `rhs.protocol_`.
- (6.5) `rhs.is_open() == false`.

-7- *Returns:* `*this`.

-8- *Remarks:* This assignment operator shall not participate in overload resolution unless `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

18.6.4 ~~basic~~-socket operations [`socket.basic.ops`]

[...]

Assuming the changes in P1322 have been applied, remove the `basic_` prefixes from -18.7- Class `basic_datagram_socket` [`socket.dgram`]:

18.7 Class template ~~basic~~-datagram_socket [`socket.dgram`]

-1- The class template ~~basic~~-datagram_socket<`Protocol`, `Executor`> is used to send and receive discrete messages of fixed maximum length.

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

template<class Protocol, class Executor>
class basic-datagram_socket : public basic-socket<Protocol, Executor>
{
public:
    // types:

    using executor_type = Executor;
    using native_handle_type = implementation-defined; // see [socket.reqmts.native]
    using protocol_type = Protocol;
    using endpoint_type = typename protocol_type::endpoint;

    template<class OtherExecutor>
    using rebind_executor =
        basic-datagram_socket<Protocol, OtherExecutor>;

    // [socket.dgram.cons], construct / copy / destroy:

    explicit basic-datagram_socket(const executor_type& ex);
    template<class ExecutionContext>
        explicit basic-datagram_socket(ExecutionContext& ctx);
    basic-datagram_socket(const executor_type& ex, const protocol_type& protocol);
    template<class ExecutionContext>
        basic-datagram_socket(ExecutionContext& ctx, const protocol_type& protocol);
    basic-datagram_socket(const executor_type& ex, const endpoint_type& endpoint);
    template<class ExecutionContext>
        basic-datagram_socket(ExecutionContext& ctx, const endpoint_type& endpoint);
    basic-datagram_socket(const executor_type& ex, const protocol_type& protocol,
        const native_handle_type& native_socket);
    template<class ExecutionContext>
        basic-datagram_socket(ExecutionContext& ctx, const protocol_type& protocol,
```

```

        const native_handle_type& native_socket);
basic_datagram_socket(const basic_datagram_socket&) = delete;
basic_datagram_socket(basic_datagram_socket&& rhs);
template<class OtherProtocol, class OtherExecutor>
    basic_datagram_socket(basic_datagram_socket<OtherProtocol, OtherExecutor>&& rhs);

~basic_datagram_socket();

basic_datagram_socket& operator=(const basic_datagram_socket&) = delete;
basic_datagram_socket& operator=(basic_datagram_socket&& rhs);
template<class OtherProtocol, class OtherExecutor>
    basic_datagram_socket& operator=(basic_datagram_socket<OtherProtocol, OtherExecutor>&& rhs);

// [socket.dgram.op], basic_datagram_socket operations:

[...]

};

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

-2- Instances of class template `basic_datagram_socket` meet the requirements of Destructible (C++2014[destructible]), MoveConstructible (C++2014[moveconstructible]), and MoveAssignable (C++2014[moveassignable]).

-3- If a program performs a synchronous operation on this socket, other than close, cancel, shutdown, send, or send_to, while there is an outstanding asynchronous read operation, the behavior is undefined.

-4- If a program performs a synchronous operation on this socket, other than close, cancel, shutdown, receive, or receive_from, while there is an outstanding asynchronous write operation, the behavior is undefined.

-5- When an operation has its effects specified as if by passing the result of `native_handle()` to a POSIX function, then the operation fails with error condition `errc::bad_file_descriptor` if `is_open() == false` at the point in the effects when the POSIX function is called.

-6- If `native_handle_type` and `basic_socket<Protocol, Executor>::native_handle_type` are both defined then they name the same type.

18.7.1 `basic_datagram_socket` constructors [socket.dgram.cons]

```
explicit basic_datagram_socket(const executor_type& ex);
```

-1- *Effects*: Initializes the base class with `basic_socket<Protocol, Executor>(ex)`.

```
template<class ExecutionContext>
    explicit basic_datagram_socket(ExecutionContext& ctx);
```

Effects: Equivalent to `basic_datagram_socket(ctx.get_executor())`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>::value` is true and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

```
basic_datagram_socket(const executor_type& ex, const protocol_type& protocol);
```

-2- *Effects*: Initializes the base class with `basic_socket<Protocol, Executor>(ctx, protocol)`.

```
template<class ExecutionContext>
    explicit basic_datagram_socket(ExecutionContext& ctx, const protocol_type& protocol);
```

Effects: Equivalent to `basic_datagram_socket(ctx.get_executor(), protocol)`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>::value` is true and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

```
basic_datagram_socket(const executor_type& ex, const endpoint_type& endpoint);
```

-3- *Effects*: Initializes the base class with `basic_socket<Protocol, Executor>(ctx, endpoint)`.

```
template<class ExecutionContext>
    explicit basic_datagram_socket(ExecutionContext& ctx, const endpoint_type& endpoint);
```

Effects: Equivalent to `basic_datagram_socket(ctx.get_executor(), endpoint)`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>::value` is true and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

```
basic_datagram_socket(io_context& ctx, const protocol_type& protocol,
    const native_handle_type& native_socket);
```

-4- *Effects*: Initializes the base class with `basic_socket<Protocol, Executor>(ctx, protocol, native_socket)`.

```
template<class ExecutionContext>
    basic_datagram_socket(ExecutionContext& ctx, const protocol_type& protocol,
        const native_handle_type& native_socket);
```

Effects: Equivalent to `basic_datagram_socket(ctx.get_executor(), protocol, native_socket)`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>::value` is true and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

```
basic_datagram_socket(basic_datagram_socket&& rhs);
```

-5- *Effects*: Move constructs an object of class `basic_datagram_socket<Protocol, Executor>`, initializing the base class with `basic_socket<Protocol, Executor>(std::move(rhs))`.

```
template<class OtherProtocol, class OtherExecutor>
    basic_datagram_socket(basic_datagram_socket<OtherProtocol, OtherExecutor>&& rhs);
```

-6- *Requires*: `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

-7- *Effects*: Move constructs an object of class `basic_datagram_socket<Protocol, Executor>`, initializing the base class with `basic_socket<Protocol, Executor>(std::move(rhs))`.

-8- *Remarks*: This constructor shall not participate in overload resolution unless `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

18.7.2 `basic_datagram_socket` assignment [`socket.dgram.assign`]

```
basic_datagram_socket& operator=(basic_datagram_socket&& rhs);
```

-1- *Effects*: Equivalent to `basic_socket<Protocol, Executor>::operator=(std::move(rhs))`.

-2- *Returns*: `*this`.

```
template<class OtherProtocol, class OtherExecutor>
    basic_datagram_socket& operator=(basic_datagram_socket<OtherProtocol, OtherExecutor>&& rhs);
```

-3- *Requires*: `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

-4- *Effects*: Equivalent to `basic_socket<Protocol, Executor>::operator=(std::move(rhs))`.

-5- *Returns*: `*this`.

-6- *Remarks*: This assignment operator not participate in overload resolution unless `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

18.7.3 `basic_datagram_socket` operations [`socket.dgram.op`]

[...]

Assuming the changes in P1322 have been applied, remove the `basic_` prefixes from -18.8- Class `basic_stream_socket` [`socket.stream`]:

18.8 Class template `basic_stream_socket` [`socket.stream`]

-1- The class template `basic_stream_socket<Protocol, Executor>` is used to exchange data with a peer over a sequenced, reliable, bidirectional, connection-mode byte stream.

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

template<class Protocol, class Executor>
class basic_stream_socket : public basic_socket<Protocol, Executor>
{
public:
    // types:

    using executor_type = Executor;
    using native_handle_type = implementation-defined; // see [socket.reqmts.native]
    using protocol_type = Protocol;
    using endpoint_type = typename protocol_type::endpoint;

    template<class OtherExecutor>
    using rebind_executor =
        basic_stream_socket<Protocol, OtherExecutor>;

    // [socket.stream.cons], construct / copy / destroy:

    explicit basic_stream_socket(const executor_type& ex);
    template<class ExecutionContext>
        explicit basic_stream_socket(ExecutionContext& ctx);
    basic_stream_socket(const executor_type& ex, const protocol_type& protocol);
    template<class ExecutionContext>
        basic_stream_socket(ExecutionContext& ctx, const protocol_type& protocol);
    basic_stream_socket(const executor_type& ex, const endpoint_type& endpoint);
    template<class ExecutionContext>
        basic_stream_socket(ExecutionContext& ctx, const endpoint_type& endpoint);
    basic_stream_socket(const executor_type& ex, const protocol_type& protocol,
        const native_handle_type& native_socket);
    template<class ExecutionContext>
        basic_stream_socket(ExecutionContext& ctx, const protocol_type& protocol,
            const native_handle_type& native_socket);
    basic_stream_socket(const basic_stream_socket&) = delete;
    basic_stream_socket(basic_stream_socket&& rhs);
    template<class OtherProtocol, class OtherExecutor>
        basic_stream_socket(basic_stream_socket<OtherProtocol, OtherExecutor>&& rhs);

    ~basic_stream_socket();

    basic_stream_socket& operator=(const basic_stream_socket&) = delete;
    basic_stream_socket& operator=(basic_stream_socket&& rhs);
    template<class OtherProtocol, class OtherExecutor>
        basic_stream_socket& operator=(basic_stream_socket<OtherProtocol, OtherExecutor>&& rhs);

    // [socket.stream.ops], basic_stream_socket operations:

    [...]

};

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std
```

-2- Instances of class template `basic_stream_socket` meet the requirements of `Destructible` (C++2014[`destructible`]), `MoveConstructible` (C++2014[`moveconstructible`]), `MoveAssignable` (C++2014[`moveassignable`]), `SyncReadStream` ([`buffer.stream.reqmts.syncreadstream`]), `SyncWriteStream` ([`buffer.stream.reqmts.syncwritestream`]), `AsyncReadStream` ([`buffer.stream.reqmts.asyncreadstream`]), and `AsyncWriteStream` ([`buffer.stream.reqmts.asyncwritestream`]).

-3- If a program performs a synchronous operation on this socket, other than close, cancel, shutdown, or send, while there is an outstanding asynchronous read operation, the behavior is undefined.

-4- If a program performs a synchronous operation on this socket, other than close, cancel, shutdown, or receive, while there is an outstanding asynchronous write operation, the behavior is undefined.

-5- When an operation has its effects specified as if by passing the result of `native_handle()` to a POSIX function, then the operation fails with error condition `errc::bad_file_descriptor` if `is_open() == false` at the point in the effects when the POSIX function is called.

-6- If `native_handle_type` and `basic_socket<Protocol, Executor>::native_handle_type` are both defined then they name the same type.

18.8.1 `basic_stream_socket` constructors [`socket.stream.cons`]

```
explicit basic_stream_socket(const executor_type& ex);
```

-1- *Effects*: Initializes the base class with `basic_socket<Protocol, Executor>(ctx)`.

```
template<class ExecutionContext>
    explicit basic_stream_socket(ExecutionContext& ctx);
```

Effects: Equivalent to `basic_stream_socket(ctx.get_executor())`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>::value` is true and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

```
basic_stream_socket(const executor_type& ex, const protocol_type& protocol);
```

-2- *Effects*: Initializes the base class with `basic_socket<Protocol, Executor>(ctx, protocol)`.

```
template<class ExecutionContext>
    explicit basic_stream_socket(ExecutionContext& ctx, const protocol_type& protocol);
```

Effects: Equivalent to `basic_stream_socket(ctx.get_executor(), protocol)`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>::value` is true and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

```
basic_stream_socket(const executor_type& ex, const endpoint_type& endpoint);
```

-3- *Effects*: Initializes the base class with `basic_socket<Protocol, Executor>(ctx, endpoint)`.

```
template<class ExecutionContext>
    explicit basic_stream_socket(ExecutionContext& ctx, const endpoint_type& endpoint);
```

Effects: Equivalent to `basic_stream_socket(ctx.get_executor(), endpoint)`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>::value` is true and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

```
basic_stream_socket(const executor_type& ex, const protocol_type& protocol,
                    const native_handle_type& native_socket);
```

-4- *Effects*: Initializes the base class with `basic_socket<Protocol, Executor>(ctx, protocol, native_socket)`.

```
template<class ExecutionContext>
    basic_stream_socket(ExecutionContext& ctx, const protocol_type& protocol,
                       const native_handle_type& native_socket);
```

Effects: Equivalent to `basic_stream_socket(ctx.get_executor(), protocol, native_socket)`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>::value` is true and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.


```
basic_stream_socket(basic_stream_socket&& rhs);
```

-5- *Effects*: Move constructs an object of class `basic_stream_socket<Protocol, Executor>`, initializing the base class with `basic_socket<Protocol, Executor>(std::move(rhs))`.

```
template<class OtherProtocol, class OtherExecutor>
    basic_stream_socket(basic_stream_socket<OtherProtocol, OtherExecutor>&& rhs);
```

-6- *Requires*: `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

-7- *Effects*: Move constructs an object of class `basic_stream_socket<Protocol, Executor>`, initializing the base class with `basic_socket<Protocol, Executor>(std::move(rhs))`.

-8- *Remarks*: This constructor shall not participate in overload resolution unless `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

18.8.2 `basic_stream_socket` assignment [socket.stream.assign]

```
basic_stream_socket& operator=(basic_stream_socket&& rhs);
```

-1- *Effects*: Equivalent to `basic_socket<Protocol, Executor>::operator=(std::move(rhs))`.

-2- *Returns*: `*this`.

```
template<class OtherProtocol, class OtherExecutor>
    basic_stream_socket& operator=(basic_stream_socket<OtherProtocol, OtherExecutor>&& rhs);
```

-3- *Requires*: `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

-4- *Effects*: Equivalent to `basic_socket<Protocol, Executor>::operator=(std::move(rhs))`.

-5- *Returns*: `*this`.

-6- *Remarks*: This assignment operator shall not participate in overload resolution unless `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

18.8.3 `basic_stream_socket` operations [socket.stream.ops]

[...]

Assuming the changes in P1322 have been applied, remove the `basic_` prefixes from -18.9- Class `basic_socket_acceptor` [socket.acceptor]:

18.9 Class template `basic_socket_acceptor` [socket.acceptor]

-1- An object of class template `basic_socket_acceptor<AcceptableProtocol, Executor>` is used to listen for, and queue, incoming socket connections. Socket objects that represent the incoming connections are dequeued by calling `accept` or `async_accept`.

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

    template<class AcceptableProtocol, class Executor>
    class basic_socket_acceptor : public socket_base
    {
    public:
        // types:

        using executor_type = Executor;
        using native_handle_type = implementation-defined; // see [socket.reqmts.native]
        using protocol_type = AcceptableProtocol;
        using endpoint_type = typename protocol_type::endpoint;
        using socket_type = typename protocol_type::socket_for<executor_type>;
        template<class OtherExecutor>
```

```

using socket_type_for
    = typename protocol_type::socket_for<OtherExecutor>;

template<class OtherExecutor>
using rebind_executor =
    basic_socket_acceptor<AcceptableProtocol, OtherExecutor>;

// [socket.acceptor.cons], construct / copy / destroy:

explicit basic_socket_acceptor(const executor_type& ex);
template<class ExecutionContext>
    explicit basic_socket_acceptor(ExecutionContext& ctx);
basic_socket_acceptor(const executor_type& ex, const protocol_type& protocol);
template<class ExecutionContext>
    basic_socket_acceptor(ExecutionContext& ctx, const protocol_type& protocol);
basic_socket_acceptor(const executor_type& ex, const endpoint_type& endpoint,
    bool reuse_addr = true);
template<class ExecutionContext>
    basic_socket_acceptor(ExecutionContext& ctx, const endpoint_type& endpoint,
    bool reuse_addr = true);
basic_socket_acceptor(const executor_type& ex, const protocol_type& protocol,
    const native_handle_type& native_socket);
template<class ExecutionContext>
    basic_socket_acceptor(ExecutionContext& ctx, const protocol_type& protocol,
    const native_handle_type& native_socket);
basic_socket_acceptor(const basic_socket_acceptor&) = delete;
basic_socket_acceptor(basic_socket_acceptor&& rhs);
template<class OtherProtocol, class OtherExecutor>
    basic_socket_acceptor(basic_socket_acceptor<OtherProtocol, OtherExecutor>&& rhs);

~basic_socket_acceptor();

basic_socket_acceptor& operator=(const basic_socket_acceptor&) = delete;
basic_socket_acceptor& operator=(basic_socket_acceptor&& rhs);
template<class OtherProtocol, class OtherExecutor>
    basic_socket_acceptor& operator=(basic_socket_acceptor<OtherProtocol, OtherExecutor>&& rhs);

// [socket.acceptor.ops], basic_socket_acceptor operations:

[...];

};

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

-2- Instances of class template ~~basic~~_socket_acceptor meet the requirements of Destructible (C++2014[destructible]), MoveConstructible (C++2014[moveconstructible]), and MoveAssignable (C++2014[moveassignable]).

-3- When there are multiple outstanding asynchronous accept operations the order in which the incoming connections are dequeued, and the order of invocation of the completion handlers for these operations, is unspecified.

-4- When an operation has its effects specified as if by passing the result of native_handle() to a POSIX function, then the operation fails with error condition errc::bad_file_descriptor if is_open() == false at the point in the effects when the POSIX function is called.

18.9.1 ~~basic~~_socket_acceptor constructors [socket.acceptor.cons]

```
explicit basic_socket_acceptor(const executor_type& ex);
```

-1- *Postconditions*:

- (1.1) get_executor() == ex.
- (1.2) is_open() == false.

```
template<class ExecutionContext>
    explicit basic_socket_acceptor(ExecutionContext& ctx);
```

Effects: Equivalent to ~~basic~~_socket_acceptor(ctx.get_executor()).

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context&>::value` is true and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

```
basic_socket_acceptor(const executor_type& ex, const protocol_type& protocol);
```

-2- *Effects:* Opens this acceptor as if by calling `open(protocol)`.

-3- *Postconditions:*

- (3.1) `get_executor() == ex`.
- (3.2) `is_open() == true`.
- (3.3) `non_blocking() == false`.
- (3.4) `enable_connection_aborted() == false`.
- (3.5) `protocol_ == protocol`.

```
template<class ExecutionContext>  
explicit basic_socket_acceptor(ExecutionContext& ctx, const protocol_type& protocol);
```

Effects: Equivalent to `basic_socket_acceptor(ctx.get_executor(), protocol)`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context&>::value` is true and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

```
basic_socket_acceptor(const executor_type& ex, const endpoint_type& endpoint,  
                      bool reuse_addr = true);
```

-4- *Effects:* Opens and binds this acceptor as if by calling:

```
open(endpoint.protocol());  
if (reuse_addr)  
    set_option(reuse_address(true));  
bind(endpoint);  
listen();
```

-5- *Postconditions:*

- (5.1) `get_executor() == ex`.
- (5.2) `is_open() == true`.
- (5.3) `non_blocking() == false`.
- (5.4) `enable_connection_aborted() == false`.
- (5.5) `protocol_ == endpoint.protocol()`.

```
template<class ExecutionContext>  
explicit basic_socket_acceptor(ExecutionContext& ctx, const endpoint_type& endpoint,  
                              bool reuse_addr = true);
```

Effects: Equivalent to `basic_socket_acceptor(ctx.get_executor(), endpoint, reuse_addr)`.

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context&>::value` is true and `is_constructible<executor_type, typename ExecutionContext::executor_type>::value` is true.

```
basic_socket_acceptor(const executor_type& ex, const protocol_type& protocol,  
                      const native_handle_type& native_acceptor);
```

-6- *Requires:* `native_acceptor` is a native handle to an open acceptor.

-7- *Effects:* Assigns the existing native acceptor into this acceptor as if by calling `assign(protocol, native_acceptor)`.

-8- *Postconditions:*

- (8.1) `get_executor() == ex`.
- (8.2) `is_open() == true`.
- (8.3) `non_blocking() == false`.
- (8.4) `enable_connection_aborted() == false`.
- (8.5) `protocol_ == protocol`.

```
template<class ExecutionContext>
    basic_socket_acceptor(ExecutionContext& ctx, const protocol_type& protocol,
                           const native_handle_type& native_socket);
```

Effects: Equivalent to ~~basic~~_socket_acceptor(ctx.get_executor(), protocol, native_socket).

Remarks: This function shall not participate in overload resolution unless `is_convertible<ExecutionContext&, execution_context_t::value>` is true and `is_constructible<executor_type, typename ExecutionContext::executor_type>` is true.

```
basic_socket_acceptor(basic_socket_acceptor&& rhs);
```

-9- *Effects:* Move constructs an object of class ~~basic~~_socket_acceptor<AcceptableProtocol, Executor> that refers to the state originally represented by rhs.

-10- *Postconditions:*

- (10.1) `get_executor()` == `rhs.get_executor()`.
- (10.2) `is_open()` returns the same value as `rhs.is_open()` prior to the constructor invocation.
- (10.3) `non_blocking()` returns the same value as `rhs.non_blocking()` prior to the constructor invocation.
- (10.4) `enable_connection_aborted()` returns the same value as `rhs.enable_connection_aborted()` prior to the constructor invocation.
- (10.5) `native_handle()` returns the same value as `rhs.native_handle()` prior to the constructor invocation.
- (10.6) `protocol_` is equal to the prior value of `rhs.protocol_`.
- (10.7) `rhs.is_open()` == false.

```
template<class OtherProtocol, class OtherExecutor>
    basic_socket_acceptor(basic_socket_acceptor<OtherProtocol, OtherExecutor>&& rhs);
```

-11- *Requires:* OtherProtocol is implicitly convertible to Protocol and OtherExecutor is implicitly convertible to Executor.

-12- *Effects:* Move constructs an object of class ~~basic~~_socket_acceptor<AcceptableProtocol, Executor> that refers to the state originally represented by rhs.

-13- *Postconditions:*

- (13.1) `get_executor()` == `rhs.get_executor()`.
- (13.2) `is_open()` returns the same value as `rhs.is_open()` prior to the constructor invocation.
- (13.3) `non_blocking()` returns the same value as `rhs.non_blocking()` prior to the constructor invocation.
- (13.4) `enable_connection_aborted()` returns the same value as `rhs.enable_connection_aborted()` prior to the constructor invocation.
- (13.5) `native_handle()` returns the prior value of `rhs.native_handle()`.
- (13.6) `protocol_` is the result of converting the prior value of `rhs.protocol_`.
- (13.7) `rhs.is_open()` == false.

-14- *Remarks:* This constructor shall not participate in overload resolution unless OtherProtocol is implicitly convertible to Protocol and OtherExecutor is implicitly convertible to Executor.

18.9.2 ~~basic~~_socket_acceptor destructor [socket.acceptor.dtor]

```
~basic_socket_acceptor();
```

-1- *Effects:* If `is_open()` is true, cancels all outstanding asynchronous operations associated with this acceptor, and releases acceptor resources as if by POSIX `close(native_handle())`. Completion handlers for canceled operations are passed an error code `ec` such that `ec == errc::operation_canceled` yields true.

18.9.3 ~~basic~~_socket_acceptor assignment [socket.acceptor.assign]

```
basic_socket_acceptor& operator=(basic_socket_acceptor&& rhs);
```

-1- *Effects:* If `is_open()` is true, cancels all outstanding asynchronous operations associated with this acceptor, and releases acceptor resources as if by POSIX `close(native_handle())`. Then moves into `*this` the state originally represented by rhs. Completion handlers for canceled operations are passed an error code `ec` such that `ec == errc::operation_canceled` yields true.

-2- *Postconditions:*

- (2.1) `get_executor()` == `rhs.get_executor()`.
- (2.2) `is_open()` returns the same value as `rhs.is_open()` prior to the assignment.
- (2.3) `non_blocking()` returns the same value as `rhs.non_blocking()` prior to the assignment.
- (2.4) `enable_connection_aborted()` returns the same value as `rhs.enable_connection_aborted()` prior to the assignment.
- (2.5) `native_handle()` returns the same value as `rhs.native_handle()` prior to the assignment.
- (2.6) `protocol_` is the same value as `rhs.protocol_` prior to the assignment.
- (2.7) `rhs.is_open()` == `false`.

-3- *Returns:* `*this`.

```
template<class OtherProtocol, class OtherExecutor>
    basic_socket_acceptor& operator=(basic_socket_acceptor<OtherProtocol, OtherExecutor>&& rhs);
```

-4- *Requires:* `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

-5- *Effects:* If `is_open()` is true, cancels all outstanding asynchronous operations associated with this acceptor, and releases acceptor resources as if by `POSIX close(native_handle())`. Then moves into `*this` the state originally represented by `rhs`. Completion handlers for canceled operations are passed an error code `ec` such that `ec == errc::operation_canceled` yields true.

-6- *Postconditions:*

- (6.1) `get_executor()` == `rhs.get_executor()`.
- (6.2) `is_open()` returns the same value as `rhs.is_open()` prior to the assignment.
- (6.3) `non_blocking()` returns the same value as `rhs.non_blocking()` prior to the assignment.
- (6.4) `enable_connection_aborted()` returns the same value as `rhs.enable_connection_aborted()` prior to the assignment.
- (6.5) `native_handle()` returns the same value as `rhs.native_handle()` prior to the assignment.
- (6.6) `protocol_` is the result of converting the value of `rhs.protocol_` prior to the assignment.
- (6.7) `rhs.is_open()` == `false`.

-7- *Returns:* `*this`.

-8- *Remarks:* This assignment operator shall not participate in overload resolution unless `OtherProtocol` is implicitly convertible to `Protocol` and `OtherExecutor` is implicitly convertible to `Executor`.

18.9.4 ~~basic~~_socket_acceptor operations [socket.acceptor.ops]

[...]

Assuming the changes in P1322 have been applied,

- use `io_context::executor_type` as the default for the `iostream` class templates' `Executor` template parameters,
- use the `protocol_type`'s `resolver_for` type alias instead of `resolver`, and
- remove the `basic_` prefixes

in -19.1- Class template `basic_socket_streambuf` [socket.streambuf]:

19.1 Class template ~~basic~~_socket_streambuf [socket.streambuf]

-1- The class ~~basic~~_socket_streambuf<Protocol, Clock, WaitTraits, Executor> associates both the input sequence and the output sequence with a socket. The input and output sequences do not support seeking. [*Note:* The input and output sequences are independent as a stream socket provides full duplex I/O. -- end note]

-2- [*Note:* This class is intended for sending and receiving bytes, not characters. Any conversion from characters to bytes, and vice versa, occurs elsewhere. -- end note]

```
namespace std {
namespace experimental {
namespace net {
```

```

inline namespace v1 {

template<class Protocol, class Clock, class WaitTraits, class Executor = executorio_context::executor_type>
class basic_socket_streambuf : public basic_streambuf<char>
{
public:
    // types:

    using executor_type = Executor;
    using protocol_type = Protocol;
    using endpoint_type = typename protocol_type::endpoint;
    using clock_type = Clock;
    using time_point = typename clock_type::time_point;
    using duration = typename clock_type::duration;
    using wait_traits_type = WaitTraits;

    // [socket.streambuf.cons], construct / copy / destroy:

    basic_socket_streambuf();
    explicit basic_socket_streambuf(basic_stream_socket<protocol_type, executor_type> s);
    basic_socket_streambuf(const basic_socket_streambuf&) = delete;
    basic_socket_streambuf(basic_socket_streambuf&& rhs);

    virtual ~basic_socket_streambuf();

    basic_socket_streambuf& operator=(const basic_socket_streambuf&) = delete;
    basic_socket_streambuf& operator=(basic_socket_streambuf&& rhs);

    // [socket.streambuf.members], members:

    basic_socket_streambuf* connect(const endpoint_type& e);
    template<class... Args> basic_socket_streambuf* connect(Args&&... );

    basic_socket_streambuf* close();

    basic_socket<protocol_type, executor_type>& socket();
    error_code error() const;

    time_point expiry() const;
    void expires_at(const time_point& t);
    void expires_after(const duration& d);

protected:
    // overridden virtual functions:
    virtual int_type underflow() override;
    virtual int_type pbackfail(int_type c = traits_type::eof()) override;
    virtual int_type overflow(int_type c = traits_type::eof()) override;
    virtual int sync() override;
    virtual streambuf* setbuf(char_type* s, streamsize n) override;

private:
    basic_stream_socket<protocol_type, executor_type> socket_; // exposition only
    error_code ec_; // exposition only
    time_point expiry_; // exposition only
};

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

-3- Instances of class template ~~basic~~_socket_streambuf meet the requirements of Destructible (C++2014[destructible]), MoveConstructible (C++2014[moveconstructible]), and MoveAssignable (C++2014[moveassignable]).

19.1.1 ~~basic~~_socket_streambuf constructors [socket.streambuf.cons]

~~basic~~_socket_streambuf();

-1- *Effects*: Initializes socket_ with ctx, where ctx is an unspecified object of class io_context.

-2- *Postconditions*: expiry() == time_point::max() and !error().

*Remarks:*This function shall not participate in overload resolution unless `is_constructible<executor_type, io_context::executor_type>::value` is true.

```
explicit basic_socket_streambuf(basic_stream_socket<protocol_type, executor_type> s);
```

-3- *Effects:*Initializes `socket_` with `std::move(s)`.

-4- *Postconditions:*`expiry() == time_point::max()` and `!error()`.

```
basic_socket_streambuf(basic_socket_streambuf&& rhs);
```

-5- *Effects:*Move constructs from the rvalue `rhs`. It is implementation-defined whether the sequence pointers in `*this` (`eback()`, `gptr()`, `egptr()`, `pbase()`, `pptr()`, `epptr()`) obtain the values which `rhs` had. Whether they do or not, `*this` and `rhs` reference separate buffers (if any at all) after the construction. Additionally `*this` references the socket which `rhs` did before the construction, and `rhs` references no open socket after the construction.

-6- *Postconditions:*Let `rhs_p` refer to the state of `rhs` just prior to this construction and let `rhs_a` refer to the state of `rhs` just after this construction.

- (6.1) `is_open() == rhs_p.is_open()`
- (6.2) `rhs_a.is_open() == false`
- (6.3) `error() == rhs_p.error()`
- (6.4) `!rhs_a.error()`
- (6.5) `expiry() == rhs_p.expiry()`
- (6.6) `rhs_a.expiry() == time_point::max()`
- (6.7) `gptr() - eback() == rhs_p.gptr() - rhs_p.eback()`
- (6.8) `egptr() - eback() == rhs_p.egptr() - rhs_p.eback()`
- (6.9) `ptr() - pbase() == rhs_p.pptr() - rhs_p.pbase()`
- (6.10) `pptr() - pbase() == rhs_p.epptr() - rhs_p.pbase()`
- (6.11) `if (eback()) eback() != rhs_a.eback()`
- (6.12) `if (gptr()) gptr() != rhs_a.gptr()`
- (6.13) `if (egptr()) egptr() != rhs_a.egptr()`
- (6.14) `if (pbase()) pbase() != rhs_a.pbase()`
- (6.15) `if (pptr()) pptr() != rhs_a.pptr()`
- (6.16) `if (epptr()) epptr() != rhs_a.epptr()`

```
virtual ~basic_socket_streambuf();
```

-7- *Effects:*If a put area exists, calls `overflow(traits_type::eof())` to flush characters. [*Note:* The socket is closed by the `basic_stream_socket<protocol_type, executor_type>` destructor. -- end note]

```
basic_socket_streambuf& operator=(basic_socket_streambuf&& rhs);
```

-8- *Effects:*Calls `this->close()` then move assigns from `rhs`. After the move assignment `*this` and `rhs` have the observable state they would have had if `*this` had been move constructed from `rhs`.

-9- *Returns:*`*this`.

19.1.2 `basic_socket_streambuf` members [socket.streambuf.members]

```
basic_socket_streambuf* connect(const endpoint_type& e);
```

-1- *Effects:*Initializes the `basic_socket_streambuf` as required, closes and re-opens the socket by performing `socket_.close(ec_)` and `socket_.open(e.protocol(), ec_)`, then attempts to establish a connection as if by POSIX `connect(socket_.native_handle(), static_cast<sockaddr*>(e.data()), e.size())`. `ec_` is set to reflect the error code produced by the operation. If the operation does not complete before the absolute timeout specified by `expiry_`, the socket is closed and `ec_` is set to `errc::timed_out`.

-2- *Returns:*if `!ec_`, `this`; otherwise, a null pointer.

```
template<class... Args>  
basic_socket_streambuf* connect(Args&&... args);
```

-3- *Effects:*Initializes the `basic_socket_streambuf` as required and closes the socket as if by calling `socket_.close(ec_)`. Obtains an endpoint sequence endpoints by performing `protocol_type::resolver_for<executor_type>(ctx).resolve(forward<Args>(args)...) , where ctx is an unspecified object of class io_context. For each endpoint e in the`

sequence, closes and re-opens the socket by performing `socket_.close(ec_)` and `socket_.open(e.protocol(), ec_)`, then attempts to establish a connection as if by POSIX `connect(socket_.native_handle(), static_cast<sockaddr*>(e.data()), e.size())`. `ec_` is set to reflect the error code produced by the operation. If the operation does not complete before the absolute timeout specified by `expiry_`, the socket is closed and `ec_` is set to `errc::timed_out`.

-4- *Returns*:if `!ec_`, this; otherwise, a null pointer.

-5- *Remarks*:This function shall not participate in overload resolution unless `Protocol` meets the requirements for an internet protocol ([`internet.reqmts.protocol`]).

basic_socket_streambuf* `close()`;

-6- *Effects*:If a put area exists, calls `overflow(traits_type::eof())` to flush characters. Regardless of whether the preceding call fails or throws an exception, the function closes the socket as if by **basic_socket**<`protocol_type`, `executor_type`>::`close(ec_)`. If any of the calls made by the function fail, `close` fails by returning a null pointer. If one of these calls throws an exception, the exception is caught and rethrown after closing the socket.

-7- *Returns*:this on success, a null pointer otherwise.

-8- *Postconditions*:`is_open() == false`.

basic_socket<`protocol_type`, `executor_type`>& `socket()`;

-9- *Returns*:`socket_`.

`error_code error() const`;

-10- *Returns*:`ec_`.

`time_point expiry() const`;

-11- *Returns*:`expiry_`.

`void expires_at(const time_point& t)`;

-12- *Postconditions*:`expiry_ == t`.

`void expires_after(const duration& d)`;

-13- *Effects*:Equivalent to `expires_at(clock_type::now() + d)`.

19.1.3 **basic_socket_streambuf** overridden virtual functions [`socket.streambuf.virt`]

[...]

Assuming the changes in P1322 have been applied,

- use `io_context::executor_type` as the default for the `iostream` class templates' *Executor* template parameters,
- remove the `basic_` prefixes

in -19.2- Class template **basic_socket_iostream** [`socket.iostream`]:

19.2 Class template **basic_socket_iostream** [`socket.iostream`]

-1- The class template **basic_socket_iostream**<`Protocol`, `Clock`, `WaitTraits`, `Executor`> supports reading and writing on sockets. It uses a **basic_socket_streambuf**<`Protocol`, `Clock`, `WaitTraits`, `Executor`> object to control the associated sequences.

-2- [*Note*: This class is intended for sending and receiving bytes, not characters. Any conversion from characters to bytes, and vice versa, occurs elsewhere. -- *end note*]

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {
```

```

template<class Protocol, class Clock, class WaitTraits, class Executor = executorio_context::executor_type>
class basic_socket_iostream : public basic_iostream<char>
{
public:
    // types:

    using executor_type = Executor;
    using protocol_type = Protocol;
    using endpoint_type = typename protocol_type::endpoint;
    using clock_type = Clock;
    using time_point = typename clock_type::time_point;
    using duration = typename clock_type::duration;
    using wait_traits_type = WaitTraits;

    // [socket.iostream.cons], construct / copy / destroy:

    basic_socket_iostream();
    explicit basic_socket_iostream(basic_stream_socket<protocol_type, executor_type> s);
    basic_socket_iostream(const basic_socket_iostream&) = delete;
    basic_socket_iostream(basic_socket_iostream&& rhs);
    template<class... Args>
        explicit basic_socket_iostream(Args&&... args);

    basic_socket_iostream& operator=(const basic_socket_iostream&) = delete;
    basic_socket_iostream& operator=(basic_socket_iostream&& rhs);

    // [socket.iostream.members], members:

    template<class... Args> void connect(Args&&... args);

    void close();

    basic_socket_streambuf<protocol_type, clock_type, wait_traits_type, executor_type>* rdbuf() const;

    basic_socket<protocol_type, executor_type>& socket();
    error_code error() const;

    time_point expiry() const;
    void expires_at(const time_point& t);
    void expires_after(const duration& d);

private:
    basic_socket_streambuf<protocol_type, clock_type, wait_traits_type, executor_type> sb_; // exposition only
};

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

-3- Instances of class template ~~basic~~_socket_iostream meet the requirements of Destructible (C++2014[destructible]), MoveConstructible (C++2014[moveconstructible]), and MoveAssignable (C++2014[moveassignable]).

19.2.1 ~~basic~~_socket_iostream constructors [socket.iostream.cons]

```
basic_socket_iostream();
```

-1- *Effects*: Initializes the base class as basic_iostream<char>(&sb_), value-initializes sb_, and performs setf(std::ios_base::unitbuf).

Remarks: This function shall not participate in overload resolution unless is_default_constructible<~~basic~~_socket_streambuf<protocol_type, clock_type, wait_traits_type, executor_type>>::value is true.

```
explicit basic_socket_iostream(basic_stream_socket<protocol_type, executor_type> s);
```

-2- *Effects*: Initializes the base class as basic_iostream<char>(&sb_), initializes sb_ with std::move(s), and performs setf(std::ios_base::unitbuf).

```
basic_socket_iostream(basic_socket_iostream&& rhs);
```

-3- *Effects*: Move constructs from the rvalue rhs. This is accomplished by move constructing the base class, and the contained `basic_socket_streambuf`. Next `basic_istream<char>::set_rdbuf(&sb_)` is called to install the contained `basic_socket_streambuf`.

```
template<class... Args>
    explicit basic_socket_istream(Args&&... args);
```

-4- *Effects*: Initializes the base class as `basic_istream<char>(&sb_)`, value-initializes `sb_`, and performs `setf(std::ios_base::unitbuf)`. Then calls `rdbuf()->connect(forward<Args>(args)...)` . If that function returns a null pointer, calls `setstate(failbit)`.

```
basic_socket_istream& operator=(basic_socket_istream&& rhs);
```

-5- *Effects*: Move assigns the base and members of `*this` from the base and corresponding members of rhs.

-6- *Returns*: `*this`.

19.2.2 `basic_socket_istream` members [socket.istream.members]

```
template<class... Args>
    void connect(Args&&... args);
```

-1- *Effects*: Calls `rdbuf()->connect(forward<Args>(args)...)` . If that function returns a null pointer, calls `setstate(failbit)` (which may throw `ios_base::failure`).

```
void close();
```

-2- *Effects*: Calls `rdbuf()->close()`. If that function returns a null pointer, calls `setstate(failbit)` (which may throw `ios_base::failure`).

```
basic_socket_streambuf<protocol_type, clock_type, wait_traits_type, executor_type>* rdbuf() const;
```

-3- Let SB be the type `basic_socket_streambuf<protocol_type, clock_type, wait_traits_type, executor_type>`.

-4- *Returns*: `const_cast<SB*>(addressof(sb_))`.

```
basic_socket<protocol_type, executor_type>& socket();
```

-5- *Returns*: `rdbuf()->socket()`.

[...]

Remove the `basic_` prefix from `basic_socket` in -20.1- Synchronous connect operations [socket.algo.connect] as follows:

```
template<class Protocol, class Executor, class EndpointSequence>
    typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,
                                       const EndpointSequence& endpoints);
template<class Protocol, class Executor, class EndpointSequence>
    typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,
                                       const EndpointSequence& endpoints,
                                       error_code& ec);
```

[...]

```
template<class Protocol, class Executor, class EndpointSequence, class ConnectCondition>
    typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,
                                       const EndpointSequence& endpoints,
                                       ConnectCondition c);
template<class Protocol, class Executor, class EndpointSequence, class ConnectCondition>
    typename Protocol::endpoint connect(basic_socket<Protocol, Executor>& s,
                                       const EndpointSequence& endpoints,
                                       ConnectCondition c, error_code& ec);
```

[...]

```
template<class Protocol, class Executor, class InputIterator>
    InputIterator connect(basic_socket<Protocol, Executor>& s,
                        InputIterator first, InputIterator last);
template<class Protocol, class Executor, class InputIterator>
```

```
InputIterator connect(basic_socket<Protocol, Executor>& s,
                    InputIterator first, InputIterator last,
                    error_code& ec);
```

[...]

```
template<class Protocol, class Executor, class InputIterator, class ConnectCondition>
InputIterator connect(basic_socket<Protocol, Executor>& s,
                    InputIterator first, InputIterator last,
                    ConnectCondition c);
```

```
template<class Protocol, class Executor, class InputIterator, class ConnectCondition>
InputIterator connect(basic_socket<Protocol, Executor>& s,
                    InputIterator first, InputIterator last,
                    ConnectCondition c, error_code& ec);
```

[...]

Remove the *basic_* prefix from *basic_socket* in -20.2- *Asynchronous connect operations* [*socket.algo.async.connect*] as follows:

```
template<class Protocol, class Executor, class EndpointSequence, class CompletionToken>
DEDUCED async_connect(basic_socket<Protocol, Executor>& s,
                    const EndpointSequence& endpoints,
                    CompletionToken&& token);
```

[...]

```
template<class Protocol, class Executor, class EndpointSequence,
class ConnectCondition, class CompletionToken>
DEDUCED async_connect(basic_socket<Protocol, Executor>& s,
                    const EndpointSequence& endpoints,
                    ConnectCondition c,
                    CompletionToken&& token);
```

[...]

```
template<class Protocol, class Executor, class InputIterator, class CompletionToken>
DEDUCED async_connect(basic_socket<Protocol, Executor>& s,
                    InputIterator first, InputIterator last,
                    CompletionToken&& token);
```

[...]

```
template<class Protocol, class Executor, class InputIterator,
class ConnectCondition, class CompletionToken>
DEDUCED async_connect(basic_socket<Protocol, Executor>& s,
                    InputIterator first, InputIterator last,
                    ConnectCondition c,
                    CompletionToken&& token);
```

[...]

Assuming the changes in P1322 have been applied,

- remove *executor* as the default for the *Executor* template parameter,
- remove the *basic_* prefix

in -21.1- *Header* <*experimental/internet*> synopsis [*internet.synop*]:

```
template<class InternetProtocol, class Executor = executor>
class basic_resolver;
```

Assuming the changes in P1322 have been applied,

- require a nested type alias *resolver_for* instead of *resolver*, and
- remove the *basic_* prefix

in -21.2.1- Internet protocol requirements [internet.reqmts.protocol]:

- 1- A type X meets the InternetProtocol requirements if it satisfies the requirements of AcceptableProtocol ([socket.reqmts.acceptableprotocol]), as well as the additional requirements listed below.
- 2- In the table below, a and b denote (possibly const) values of type X, and E is a type that models execution::executor (P0443R13).

Table 35: InternetProtocol requirements

expression	return type	assertion/note pre/post-conditions
X::resolver basic_ <u>for<E></u>	ip:: basic_ resolver<X, <u>E</u> >	The type of a resolver for the protocol <u>and executor</u> .
X::v4()	X	Returns an object representing the IP version 4 protocol.
X::v6()	X	Returns an object representing the IP version 6 protocol.
a == b	convertible to bool	Returns true if a and b represent the same IP protocol version, otherwise false.
a != b	convertible to bool	Returns !(a == b).

Assuming the changes in P1322 have been applied,

- remove the `basic_` prefix, and
- do a drive-by fix on the operation names `resolve` and `async_resolve`

in -21.15- Class template ip::basic_resolver_results [internet.resolver.results]:

[...]

- 3- A default-constructed `basic_resolver_results` object is empty. A non-empty results object is obtained only by calling a ~~basic_~~resolver object's waitresolve or `async_waitresolve` operations, or otherwise by copy construction, move construction, assignment, or swap from another non-empty results object.

Assuming the changes in P1322 have been applied,

- remove `executor` as the default for the `Executor` template parameter,
- remove the `basic_` prefix

in -21.17- Class template ip::basic_resolver [internet.resolver]:

21.17 Class template ip::~~basic_~~resolver [internet.resolver]

- 1- Objects of type ~~basic_~~resolver<InternetProtocol, Executor> are used to perform name resolution. Name resolution is the translation of a host name and service name into a sequence of endpoints, or the translation of an endpoint into its corresponding host name and service name.

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {
namespace ip {

    template<class InternetProtocol, class Executor==executor>
    class basic_resolver : public resolver_base
    {
    public:
        // types:

        using executor_type = Executor;
        using protocol_type = InternetProtocol;
        using endpoint_type = typename InternetProtocol::endpoint;
        using results_type = basic_resolver_results<InternetProtocol>;
```

```

template<class OtherExecutor>
using rebind_executor =
    basic_resolver<InternetProtocol, OtherExecutor>;

// [internet.resolver.cons], construct / copy / destroy:

explicit basic_resolver(const executor_type& ex);
template<class ExecutionContext> explicit basic_resolver(ExecutionContext& ctx);
basic_resolver(const basic_resolver&) = delete;
basic_resolver(basic_resolver&& rhs) noexcept;

~basic_resolver();

basic_resolver& operator=(const basic_resolver&) = delete;
basic_resolver& operator=(basic_resolver&& rhs);

// [internet.resolver.ops], basic_resolver operations:

executor_type get_executor() noexcept;

void cancel();

results_type resolve(string_view host_name, string_view service_name);
results_type resolve(string_view host_name, string_view service_name,
                    error_code& ec);
results_type resolve(string_view host_name, string_view service_name,
                    flags f);
results_type resolve(string_view host_name, string_view service_name,
                    flags f, error_code& ec);

template<class CompletionToken>
    DEDUCED async_resolve(string_view host_name, string_view service_name,
                        CompletionToken&& token);
template<class CompletionToken>
    DEDUCED async_resolve(string_view host_name, string_view service_name,
                        flags f, CompletionToken&& token);

results_type resolve(const protocol_type& protocol,
                    string_view host_name, string_view service_name);
results_type resolve(const protocol_type& protocol,
                    string_view host_name, string_view service_name,
                    error_code& ec);
results_type resolve(const protocol_type& protocol,
                    string_view host_name, string_view service_name,
                    flags f);
results_type resolve(const protocol_type& protocol,
                    string_view host_name, string_view service_name,
                    flags f, error_code& ec);

template<class CompletionToken>
    DEDUCED async_resolve(const protocol_type& protocol,
                        string_view host_name, string_view service_name,
                        CompletionToken&& token);
template<class CompletionToken>
    DEDUCED async_resolve(const protocol_type& protocol,
                        string_view host_name, string_view service_name,
                        flags f, CompletionToken&& token);

results_type resolve(const endpoint_type& e);
results_type resolve(const endpoint_type& e, error_code& ec);

template<class CompletionToken>
    DEDUCED async_resolve(const endpoint_type& e,
                        CompletionToken&& token);
};

} // namespace ip
} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

21.17.1 ip::~~basic_resolver~~ constructors [internet.resolver.cons]

```
explicit basic_resolver(const executor_type& ex);
```

-1- *Postconditions*:get_executor() == ex.

```
template<class ExecutionContext> explicit basic_resolver(ExecutionContext& ctx);
```

Postconditions:get_executor() == ctx.get_executor().

Remarks:This function shall not participate in overload resolution unless is_convertible<ExecutionContext&, execution_context_t::value>::value is true and is_constructible<executor_type, typename ExecutionContext::executor_type>::value is true.

```
basic_resolver(basic_resolver&& rhs) noexcept;
```

-2- *Effects*:Move constructs an object of class ~~basic_~~resolver<InternetProtocol, Executor> that refers to the state originally represented by rhs.

-3- *Postconditions*:get_executor() == rhs.get_executor().

21.17.2 ip::~~basic_~~resolver destructor [internet.resolver.dtor]

```
~basic_resolver();
```

-1- *Effects*:Destroys the resolver, canceling all asynchronous operations associated with this resolver as if by calling cancel().

21.17.3 ip::~~basic_~~resolver assignment [internet.resolver.assign]

```
basic_resolver& operator=(basic_resolver&& rhs);
```

-1- *Effects*:Cancels all outstanding asynchronous operations associated with *this as if by calling cancel(), then moves into *this the state originally represented by rhs.

-2- *Postconditions*:get_executor() == rhs.get_executor().

-3- *Returns*:*this.

21.17.4 ip::~~basic_~~resolver operations [internet.resolver.ops]

[...]

Assuming the changes in P1322 have been applied, update -21.19- Class ip::tcp as follows:

[...]

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {
namespace ip {

class tcp
{
public:
    // types:
    using endpoint = basic_endpoint<tcp>;
using resolver = basic_resolver<tcp>;
    template<class Executor>
    using resolver_for = resolver<tcp, Executor>;
using socket = basic_stream_socket<tcp>;
    template<class Executor>
        using socket_for = basic_stream_socket<tcp, Executor>;
using acceptor = basic_socket_acceptor<tcp>;
    template<class Executor>
    using acceptor_for = socket_acceptor<tcp, Executor>;
    using iostream = basic_socket_istream<tcp>;
```

Assuming the changes in P1322 have been applied, update -21.20- Class ip::udp as follows:


```
[...]
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {
namespace ip {

class udp
{
public:
// types:
using endpoint = basic_endpoint<udp>;
using resolver = basic_resolver<udp>;
template<class Executor>
using resolver_for = resolver<udp, Executor>;
using socket = basic_stream_socket<udp>;
template<class Executor>
using socket_for = basicdatagram_socket<udp, Executor>;

```

Specify the free functions dispatch, post, and defer in terms of the new executors model

Modify section -13.22- *Function dispatch [async.dispatch]* as follows:

-1- [Note: The function dispatch satisfies the requirements for an asynchronous operation (13.2.7), except for the requirement that the operation uses postthe execution::blocking.never property if it completes immediately. --end note]

[...]

-3- Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.
- ~~Performs `ex.dispatch(std::move(completion.completion_handler), alloc)`, where `ex` is the result of `get_associated_executor(completion.completion_handler)`, and `alloc` is the result of `get_associated_allocator(completion.completion_handler)`.~~
- Performs:

```
execution::execute(
std::prefer(ex, execution::blocking.possibly,
execution::allocator(alloc)).,
std::move(completion.completion_handler)).;
```

where `ex` is the result of `get_associated_executor(completion.completion_handler)`, and `alloc` is the result of `get_associated_allocator(completion.completion_handler)`.

[...]

-6- Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.
- Constructs a function object `f` containing as members:
 - an executor object `w` for the completion handler's associated executor, initialized with `std::prefer(get_associated_executor(h), execution::outstanding_work)`,
 - a copy of the completion handler `h`, initialized with `std::move(completion.completion_handler)`,
 - ~~an executor_work_guard object `w` for the completion handler's associated executor, initialized with `make_work_guard(h)`;~~

and where the effect of `f()` is:

- ~~`w.get_executor().dispatch(std::move(h), alloc)`, where `alloc` is the result of `get_associated_allocator(h)`, followed by~~
- ~~`w.reset()`;~~

```
execution::execute(
std::prefer(w, execution::blocking.possibly,
execution::allocator(alloc)).,
std::move(h)).;
```

where `alloc` is the result of `get_associated_allocator(h)`.

- ~~Performs `ex.dispatch(std::move(f), alloc)`, where `alloc` is the result of `get_associated_allocator(completion.completion_handler)` prior to the construction of `f`.~~

- Performs:

```
execution::execute(
std::prefer(ex, execution::blocking.possibly),
execution::allocator(alloc)),
std::move(f));
```

Modify section -13.23- Function post [async.post] as follows:

-3- Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.
- ~~Performs `ex.post(std::move(completion.completion_handler), alloc)`, where `ex` is the result of `get_associated_executor(completion.completion_handler)`, and `alloc` is the result of `get_associated_allocator(completion.completion_handler)`.~~
- Performs:

```
execution::execute(
std::prefer(
std::require(ex, execution::blocking.never),
execution::relationship.fork, execution::allocator(alloc)),
std::move(completion.completion_handler));
```

where `ex` is the result of `get_associated_executor(completion.completion_handler)`, and `alloc` is the result of `get_associated_allocator(completion.completion_handler)`.

[...]

-6- Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.
- Constructs a function object `f` containing as members:

- an executor object `w` for the completion handler's associated executor, initialized with `std::prefer(get_associated_executor(h), execution::outstanding_work)`,
- a copy of the completion handler `h`, initialized with `std::move(completion.completion_handler)`,
- ~~an executor_work_guard object `w` for the completion handler's associated executor, initialized with `make_work_guard(h)`;~~

and where the effect of `f()` is:

- ~~`w.get_executor().dispatch(std::move(h), alloc)`, where `alloc` is the result of `get_associated_allocator(h)`, followed by~~
- ~~`w.reset()`;~~

```
execution::execute(
std::prefer(w, execution::blocking.possibly),
execution::allocator(alloc)),
std::move(h));
```

where `alloc` is the result of `get_associated_allocator(h)`.

- ~~Performs `ex.dispatch(std::move(f), alloc)`, where `alloc` is the result of `get_associated_allocator(completion.completion_handler)` prior to the construction of `f`.~~
- Performs:

```
execution::execute(
std::prefer(
std::require(ex, execution::blocking.never),
execution::relationship.fork, execution::allocator(alloc)),
std::move(f));
```

Modify section -13.24- Function defer [async.defer] as follows:

-1- [Note: The function `defer` satisfies the requirements for an asynchronous operation (13.2.7), ~~except for the requirement that the operation uses `post` if it completes immediately.~~ --end note]

[...]

-3- Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.

- ~~Performs `ex.defer(std::move(completion.completion_handler), alloc)`, where `ex` is the result of `get_associated_executor(completion.completion_handler)`, and `alloc` is the result of `get_associated_allocator(completion.completion_handler)`;~~

- Performs:

```
    execution::execute(
    std::prefer(
        std::require(ex, execution::blocking.never),
        execution::relationship.continuation, execution::allocator(alloc)),
    std::move(completion.completion_handler));
```

where `ex` is the result of `get_associated_executor(completion.completion_handler)`, and `alloc` is the result of `get_associated_allocator(completion.completion_handler)`.

[...]

-6- Effects:

- Constructs an object completion of type `async_completion<CompletionToken, void()>`, initialized with token.
- Constructs a function object `f` containing as members:

- an executor object `w` for the completion handler's associated executor, initialized with `std::prefer(get_associated_executor(h), execution::outstanding_work)`,
- a copy of the completion handler `h`, initialized with `std::move(completion.completion_handler)`,
- ~~an executor work guard object `w` for the completion handler's associated executor, initialized with `make_work_guard(h)`;~~

and where the effect of `f()` is:

- ~~`w.get_executor().dispatch(std::move(h), alloc)`, where `alloc` is the result of `get_associated_allocator(h)`, followed by~~
- ~~`w.reset()`;~~

```
    execution::execute(
    std::prefer(w, execution::blocking.possibly,
        execution::allocator(alloc)),
    std::move(h));
```

where `alloc` is the result of `get_associated_allocator(h)`.

- ~~Performs `ex.dispatch(std::move(f), alloc)`, where `alloc` is the result of `get_associated_allocator(completion.completion_handler)` prior to the construction of `f`;~~

- Performs:

```
    execution::execute(
    std::prefer(
        std::require(ex, execution::blocking.never),
        execution::relationship.continuation, execution::allocator(alloc)),
    std::move(f));
```

Update the strand adapter to conform to the new executors model

Modify section -13.25- Class template strand [async.strand] as follows:

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {
```

```
    template<class Executor>
    class strand
    {
    public:
```

[...]

```
    execution_context& context() const noexcept;
```

```
    void on_work_started() const noexcept;
```

```
    void on_work_finished() const noexcept;
```

```
    template<class Func, class ProtoAllocator>
```

```
    void dispatch(Func&& f, const ProtoAllocator& a) const;
```

```
    template<class Func, class ProtoAllocator>
```

```

void post(Func&& f, const ProtoAllocator& a) const;
template<class Func, class ProtoAllocator>
void defer(Func&& f, const ProtoAllocator& a) const;

template<class Property>
see below query(const Property& p) const;

template<class Property>
see below require(const Property& p) const;

template<class Property>
see below prefer(const Property& p) const;

template<class Function>
void execute(Function&& f) const;

```

[...]

-3- A strand provides guarantees of ordering and non-concurrency. Given:

- strand objects s1 and s2 such that s1 == s2
- a function object f1 ~~added~~submitted to the strand s1 ~~using post or defer, or using dispatch~~when the execution::blocking.never property is established in s1, or when s1.running_in_this_thread() is false
- a function object f2 ~~added~~submitted to the strand s2 ~~using post or defer, or using dispatch~~when the execution::blocking.never property is established in s2, or when s2.running_in_this_thread() is false

Modify section -13.25.4- strand operations [async.strand.ops] as follows:

```
bool running_in_this_thread() const noexcept;
```

-2- **Returns:** true if the current thread of execution is running a function that was submitted to the strand, or to any other strand object ~~s such that s == *this, using dispatch, post or defer~~that shares the same ordered, non-concurrent state, using execute; otherwise false. [Note: That is, the current thread of execution's call chain includes a function that was submitted to the strand. -- end note]

```
void on_work_started() const noexcept;
```

~~-4- Effects:~~ Calls inner_ex_.on_work_started();

```
void on_work_finished() const noexcept;
```

~~-5- Effects:~~ Calls inner_ex_.on_work_finished();

```

template<class Func, class ProtoAllocator>
void dispatch(Func&& f, const ProtoAllocator& a) const;

```

~~-6- Effects:~~ If running_in_this_thread() is true, calls DECAY_COPY(forward<Func>(f))(). (C++ 2014 [thread.decaycopy]); [Note: If f exits via an exception, the exception propagates to the caller of dispatch(). -- end note] Otherwise, requests invocation of f, as if by forwarding the function object f and allocator a to the executor inner_ex_, such that the guarantees of ordering and non-concurrency are met.

```

template<class Func, class ProtoAllocator>
void post(Func&& f, const ProtoAllocator& a) const;

```

~~-7- Effects:~~ Requests invocation of f, as if by forwarding the function object f and allocator a to the executor inner_ex_, such that the guarantees of ordering and non-concurrency are met.

```
template<class Func, class ProtoAllocator> void defer(Func&& f, const ProtoAllocator& a) const;
```

~~-8- Effects:~~ Requests invocation of f, as if by forwarding the function object f and allocator a to the executor inner_ex_, such that the guarantees of ordering and non-concurrency are met.

```

template<class Property>
see below query(const Property& p) const;

```

~~-?- Returns:~~ std::query(inner_ex_, p).

~~-?- Remarks:~~ Shall not participate in overload resolution unless can_query_v<const Executor&, const Property&> is true.

template<class Property>
see below require(const Property& p) const;

-?- Returns: A strand *s* of type `strand<decay_t<decltype(std::require(inner_ex, _p))>>`, where *s*.inner_ex is initialized with `std::require(inner_ex, _p)`, and sharing the same ordered, non-concurrent state as **this*.

-?- Remarks: Shall not participate in overload resolution unless `can_require_v<const Executor&, const Property&>` is true.

template<class Property>
see below prefer(const Property& p) const;

-?- Returns: A strand *s* of type `strand<decay_t<decltype(std::prefer(inner_ex, _p))>>`, where *s*.inner_ex is initialized with `std::prefer(inner_ex, _p)`, and sharing the same ordered, non-concurrent state as **this*.

-?- Remarks: Shall not participate in overload resolution unless `can_prefer_v<const Executor&, const Property&>` is true.

template<class Function>
void execute(Function&& f) const;

-?- Effects: Submits *f* to the executor *inner_ex*, such that the guarantees of ordering and non-concurrency are met.

Update the use_future completion token to conform to the new executors model

Modify section -13.26.2- *use_future_t* members [async.use.future.members] as follows:

-8- For any executor type *E*, the associated object for the associator `associated_executor<H, E>` is an executor where, for function objects executed using the executor's ~~`dispatch()`, `post()` or `defer()` functions~~ `execute()` function, any exception thrown is caught by a function object and stored in the associated shared state.

Modify section -13.26.3- *Partial class template specialization async_result for use_future_t* [async.use.future.result] as follows:

-3- The implementation specializes `associated_executor` for *F*. For function objects executed using the associated executor's ~~`dispatch()`, `post()` or `defer()` functions~~ `execute()` function, any exception thrown is caught by the executor and stored in the associated shared state.

-4- For any executor type *E*, the associated object for the associator `associated_executor<F, E>` is an executor where, for function objects executed using the executor's ~~`dispatch()`, `post()` or `defer()` functions~~ `execute()` function, any exception thrown by a function object is caught by the executor and stored in the associated shared state.

Update io_context to conform to the new executors model

Modify section -14.2- *Class io_context* [io_context.io_context] as follows:

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

    class io_context : public execution_context
    {
    public:
        // types:

        class executor_type;
        using executor_type = see below;
```

[...]

-1- The class `io_context` satisfies the `ExecutionContext` type requirements (13.2.3).

-?- executor_type is an executor type conforming to the specification for io_context executor types described below.

[...]

-4- For an object of type `io_context`, *outstanding work* is defined as the sum of:

- ~~the total number of calls to the `on_work_started` function, less the total number of calls to the `on_work_finished` function, to any executor of the `io_context`;~~
- the number of existing executor objects associated with the `io_context` for which the `execution::outstanding_work.tracked` property is established;
- the number of function objects that have been added to the `io_context` via any executor of the `io_context`, but not yet executed; and
- the number of function objects that are currently being executed by the `io_context`.

[...]

```
executor_type get_executor() noexcept;
```

-3- **Returns:** An executor that may be used for submitting function objects to the `io_context`. The returned executor has the following properties already established:

- `execution::blocking_possibly`
- `execution::relationship.fork`
- `execution::outstanding_work.untracked`
- `execution::mapping.thread`
- `execution::allocator(std::allocator<void>())`.

-?- `io_context` executors having a different set of established properties may be represented by distinct, unspecified types.

[...]

-13- **Remarks:** This function may invoke additional function objects through nested calls to the `io_context` executor's `dispatchexecute` member function. These do not count towards the return value.

[...]

-19- **Remarks:** This function may invoke additional function objects through nested calls to the `io_context` executor's `dispatchexecute` member function. These do not count towards the return value.

[...]

-25- **Remarks:** This function may invoke additional function objects through nested calls to the `io_context` executor's `dispatchexecute` member function. These do not count towards the return value.

Remove section -14.3- Class `io_context::executor_type` [`io_context.exec`] in its entirety.

After section -14.2- Class `io_context` [`io_context.io_context`] insert a new section as follows:

-14.-? `io_context` executor types

-1- Class `io-context-executor` is for exposition only. It is used to specify a bounded set of types that model `execution::executor` (P0443R13). All executor types accessible through `io_context::get_executor()`, and subsequent calls to the member function `require`, are members of this set and conform to the specification of `io-context-executor`.

[Note: An implementation may provide distinct types for executors that have different properties established. --end note]

```
namespace std {
namespace experimental {
namespace net {
inline namespace v1 {

    class io-context-executor
    {
    public:
        // construct / copy / destroy:

        io-context-executor(const io-context-executor& other) noexcept;
        io-context-executor(io-context-executor&& other) noexcept;

        io-context-executor& operator=(const io-context-executor& other) noexcept;
        io-context-executor& operator=(io-context-executor&& other) noexcept;

        // executor operations:
```

```

    see below require(execution::blocking t::possibly t) const;
    see below require(execution::blocking t::never t) const;
    see below require(execution::relationship t::fork t) const;
    see below require(execution::relationship t::continuation t) const;
    see below require(execution::outstanding work t::untracked t) const;
    see below require(execution::outstanding work t::tracked t) const;
    see below require(execution::allocator t<void>) const;
    template<class ProtoAllocator>
        see below require(const execution::allocator t<ProtoAllocator>& a) const;

    static constexpr execution::mapping t query(execution::mapping t) noexcept;
    io context& query(execution::context t) const noexcept;
    execution::blocking t query(execution::blocking t) const noexcept;
    execution::relationship t query(execution::relationship t) const noexcept;
    execution::outstanding work t query(execution::outstanding work t) const noexcept;
    see below query(execution::allocator t<void>) const noexcept;
    template<class ProtoAllocator>
        see below query(const execution::allocator t<ProtoAllocator>&) const noexcept;

    bool running_in_this_thread() const noexcept;

    template<class Function>
        void execute(Function&& f) const;
};

    bool operator==(const io-context-executor& a, const io-context-executor& b) noexcept;
    bool operator!=(const io-context-executor& a, const io-context-executor& b) noexcept;

} // inline namespace v1
} // namespace net
} // namespace experimental
} // namespace std

```

-2- Objects of type *io-context-executor* are associated with an *io context*, and function objects submitted using the *execute* member function will be executed by the *io context* from within a run function.

-14.?.1- io context executor constructors

io-context-executor(const *io-context-executor*& other) noexcept;

-1- **Postconditions:** **this == other.*

io-context-executor(*io-context-executor*&& other) noexcept;

-2- **Postconditions:** **this* is equal to the prior value of *other.*

-14.?.2- io context executor assignment

io-context-executor& operator=(const *io-context-executor*& other) noexcept;

-1- **Postconditions:** **this == other.*

-2- **Returns:** **this.*

io-context-executor& operator=(*io-context-executor*&& other) noexcept;

-3- **Postconditions:** **this* is equal to the prior value of *other.*

-4- **Returns:** **this.*

-14.?.3- io context executor operations

```

see below require(execution::blocking t::possibly t) const;
see below require(execution::blocking t::never t) const;
see below require(execution::relationship t::fork t) const;
see below require(execution::relationship t::continuation t) const;
see below require(execution::outstanding work t::untracked t) const;
see below require(execution::outstanding work t::tracked t) const;

```


-1- **Returns:** An object of an *io-context-executor* class, associated with the same *io* context as **this*, and having the requested property established. When the requested property is part of a group that is defined as a mutually exclusive set, any other properties in the group are removed from the returned executor object. All other properties of the returned executor object are identical to those of **this*.

see below `require(execution::allocator t<void>) const;`

-2- **Returns:** `require(execution::allocator(std::allocator<void>().))`.

template<class ProtoAllocator>

see below `require(const execution::allocator t<ProtoAllocator>& a) const;`

-3- **Returns:** An object of an *io-context-executor* class, associated with the same *io* context as **this*, with the `execution::allocator t<ProtoAllocator>` property established such that allocation and deallocation associated with function submission will be performed using a copy of `a.value()`. All other properties of the returned executor object are identical to those of **this*.

static constexpr `execution::mapping_t query(execution::execution::mapping_t) noexcept;`

-4- **Returns:** `true`.

`io context& query(execution::context t) const noexcept;`

-5- **Returns:** A reference to the associated *io* context object.

`execution::blocking_t query(execution::blocking_t) const noexcept;`

`execution::relationship_t query(execution::relationship_t) const noexcept;`

`execution::outstanding_work_t query(execution::outstanding_work_t) const noexcept;`

-6- **Returns:** The established value of the property for the executor object **this*.

see below `query(execution::allocator t<void>) const noexcept;`

template<class ProtoAllocator>

see below `query(const execution::allocator t<ProtoAllocator>&) const noexcept;`

-7- **Returns:** The allocator object associated with the executor, with type and value as previously established by the `execution::allocator t<ProtoAllocator>` property.

`bool running_in_this_thread() const noexcept;`

-8- **Returns:** `true` if the current thread of execution is calling a run function of the associated *io* context object. [Note: That is, the current thread of execution's call chain includes a run function. --end note]

template<class Function>

`void execute(Function&& f) const`

-9- **Effects:** Submits the function `f` for execution on the *io* context according to the `execution::executor` concept and the properties established for **this*. If `f` exits via an exception, the exception does not propagate to the caller of `execute()`, but is instead subsequently propagated to a caller of a run function for the *io* context object.

-14.?.4- *io* context executor comparisons

`bool operator==(const io-context-executor& a, const io-context-executor& b) noexcept;`

-1- **Returns:** `true` if `addressof(a.query(execution::context t)) == addressof(b.query(execution::context t))`, and `a` and `b` have identical properties, otherwise `false`.

`bool operator!=(const io-context-executor& a, const io-context-executor& b) noexcept;`

-2- **Returns:** `!(a == b)`.

Revision History

The following changes were made in revision 3 of this paper:

Note: changed sections in R3 are indicated by a pale blue background with a dark blue line on the right margin.

- Use "models" rather than "satisfies" when referring to the `execution::executor` concept.
- Clean up specifications of *system-context-executor* and *io-context-executor*.
- Remove `any_io_executor` type alias and all uses of polymorphic executors.
- Remove defaults for `Executor` template parameters.
- Removed the `basic_` prefixes from `timer`, `socket`, and `resolver` class templates.
- Removed proposed extensions to P0443.

The following changes were made in revision 2 of this paper:

- Updated to match the unified executor facilities as described in P0443R13.
- Updated to use property facilities from namespace `std` rather than `std::execution`.
- No longer propose the addition of the `context()` member function to I/O objects.
- Renamed the polymorphic wrapper type alias to `any_io_executor`.
- Added `any_io_executor` to the forward declaration header `<netfwd>`.
- Added changes to use `any_io_executor` as the default, assuming P1322 changes have been applied.
- Renamed `context_as` to `context_as_t`, and added template variable `context_as`.
- Updated `system_context` executor specification based on implementation experience.
- Updated `io_context` executor specification based on implementation experience.

The following changes were made in revision 1 of this paper:

- Updated to match the unified executor facilities as described in P0443R7.
- Added a section on deployment experience.
- Specify that `system_context` executors use distinct types when different properties are established.
- Added missing property queries on `system_context` executors.
- Changed `strand` to conditionally forward static `query()` calls to the adapted executor.
- Specify that `io_context` executors use distinct types when different properties are established.
- Added missing property queries on `io_context` executors.
- Added modifications to P0443R7 executors, including the `execution::context_as<>` property adapter.